Department of Computer Science and Engineering

KPR Institute of Engineering and Technology



B.E. - Computer Science and Engineering

Laboratory Record

U21CS504

Web Technologies Laboratory

Regulation 2021

Nov / Dec 2026

Department of Computer Science and Engineering

KPR Institute of Engineering and Technology

Name	:	_____
Roll Number	:	_____
Subject Code & Title	:	U21CS504 - Web Technologies Laboratory
Department	:	Computer Science and Engineering
Year & Semester	:	III Year / VI Semester

This is the certified record of work done by _____

Register Number _____

Faculty In-Charge **Head of the Department**

Place: Coimbatore

Date: _____

He/ She has submitted the record for the End Semester Practical Examination held on _____

Internal Examiner **External Examiner**

List of Experiments

Expr. No.	Name of the Experiments	Page No.	Marks (20)	Faculty Sign
1.	Static web page design using HTML and CSS			
2.	BMI Calculator using JavaScript			
3.	User registration form validation using JavaScript			
4.	Implementing web applications with API calls using JavaScript			
5.	Event registration form using PHP and MySQL			
6.	Web application for food delivery system using PHP and MySQL			
7.	Web application for leave management using Servlet and MySQL			
8.	Web application for online shopping using Servlet and MySQL			
9.	Mini Project: RentEase - Rental Property Management System			

Rubrics for Laboratory Experiments

Criteria / Points	Strongly Acceptable (4)	Acceptable (3)	Considerable (2)	Below Expectations (1)	Unacceptable (0)
Understanding the Problem Statement	The procedure is explained and the problem is well designed with appropriate analysis	Some programs are written accurately	Major errors in program writing	Programs are not written / wrong program written	Programs are not written / wrong program written
Completion of target in Lab	100% target completed	80% target completed	70% target completed	50% target completed	25% target completed
User Interface Design	Well designed problem with appropriate analysis	Designed the problem with analysis	Missing some experimental procedure with partial analysis	Missing major experimental details and analysis	Could not explain the program
Development / Implement	Optimal solution with appropriate results	80% of implementation done	65% of implementation done	Few procedures were explained	No proper explanation on implementation
Innovation	Most of the innovations applied clearly	Some of the innovations applied	Few innovations applied clearly	Innovations applied	None of the innovations used

Ex No: 01 08/07/2026

Static web page design using HTML and CSS

AIM:

To design and develop a multi-page static website using HTML5 and CSS3, covering a common navigation bar, semantic sectioning, tables, forms, images and responsive styling.

PROCEDURE:

1. Create a project folder named RentEase inside the web root.
2. Create the files index.html, about.html, properties.html, contact.html and css/style.css.
3. Declare the document type, language and character set in every page.
4. Build a shared navigation bar with <nav> and <a> so every page links to the others.
5. Design the home page with a banner image, an introduction and a list of highlights.
6. Design the about page with the organisation profile, vision, mission and objectives.
7. Design the properties page with a price table and listing cards.
8. Design the contact page with an address block and a contact form.
9. Apply CSS for colour, typography, spacing, borders, box shadows and hover effects.
10. Make the layout responsive with media queries and flexible grid columns.
11. Open index.html in a browser and verify links, images and styling.

HTML ELEMENTS USED:

<!DOCTYPE html>, <html>, <head>, <meta>, <title>, <link>, <body>, <header>, <nav>, <main>, <section>, <article>, <footer>, , <h1>-<h6>, <p>, <ul>, <ol>, <li>, <a>, <img>, <table>, <tr>, <th>, <td>, <form>, <label>, <input>, <textarea>, <button>

CSS PROPERTIES USED:

font-family, font-size, color, background-color, margin, padding, max-width, display, flex, grid, gap, text-align, text-decoration, border, border-radius, box-shadow, transition, hover, media queries, CSS custom properties (--var).

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>RentEase - Find your next home</title>
  <link rel="stylesheet" href="assets/css/style.css">
</head>
<body>
  <header class="site-header">
    <div class="wrap header-inner">
      <a class="brand" href="index.php">RentEase</a>
      <nav class="site-nav">
        <a href="index.php">Home</a>
        <a href="properties.php">Properties</a>
```

```
            <a href="login.php">Login</a>
          </nav>
        </div>
    </header>

    <section class="hero">
        <div class="wrap">
            <h1>Rent smarter. Live better.</h1>
            <p class="lead">Search listings, apply online, pay rent and track
              maintenance complaints in one place.</p>
        </div>
    </section>

    <footer class="site-footer">
        <div class="wrap">
            <p>&copy; 2026 RentEase. All rights reserved.</p>
        </div>
    </footer>
</body>
</html>
```

KEY CSS:

```
:root{
  --brand:#0f766e;
  --line:#dfe6ee;
  --radius:14px;
}

.header-inner{
  display:flex;
  align-items:center;
  justify-content:space-between;
  min-height:66px;
}

.site-nav{display:flex;gap:1.4rem}
.site-nav a{color:#4b6076;text-decoration:none}
.site-nav a:hover{color:var(--brand)}

.cards{
  display:grid;
  grid-template-columns:repeat(auto-fill,minmax(280px,1fr));
  gap:1.2rem;
}

@media (max-width:700px){
  .site-nav{flex-direction:column}
}
```

OUTPUT:

The home page renders with a fixed header, a full-width hero banner, a highlight strip and a three-column feature grid. On a 700 px viewport the columns collapse to a single column and the navigation stacks vertically.

RESULT:

Thus the static web pages were designed using HTML5 and CSS3 and were verified in the browser.

RUBRICS :

Rubrics	Actual Marks	Awarded Marks

Understanding (R1)	4	
Timeline (R2)	4	
Design (R3)	4	
Development (R4)	4	
Innovation (R5)	4	
Total Marks	**20**	

Ex No: 02 15/07/2026

BMI Calculator using JavaScript

AIM:

To design a Body Mass Index calculator using HTML, CSS and JavaScript that reads height and weight, computes the BMI value, classifies the result and validates the input.

PROCEDURE:

1. Create the files bmi.html and bmi.css.
2. Build a form with height (cm) and weight (kg) number inputs and a Calculate button.
3. Add a result panel that stays hidden until a valid calculation runs.
4. Attach a submit handler to the form and call preventDefault().
5. Read both values with parseFloat() and reject empty or non-numeric input.
6. Convert height from centimetres to metres and compute weight / height².
7. Round the result to one decimal place with toFixed(1).
8. Classify the value into Underweight, Normal, Overweight or Obese using standard cut-offs.
9. Write the value and the category into the result panel and colour-code the category.
10. Test with boundary inputs, zero, negative values and non-numeric text.

JAVASCRIPT FEATURES USED:

addEventListener, preventDefault, parseFloat, isNaN, toFixed, template literals, classList, textContent, conditional (ternary) expressions.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>BMI Calculator</title>
  <link rel="stylesheet" href="bmi.css">
</head>
<body>
  <div class="card">
    <h1>BMI Calculator</h1>

    <form id="bmiForm">
      <label for="height">Height (cm)</label>
      <input type="number" id="height" min="50" max="250" step="0.1" required>

      <label for="weight">Weight (kg)</label>
      <input type="number" id="weight" min="10" max="300" step="0.1" required>

      <button type="submit">Calculate</button>
    </form>

    <div id="result" class="result hidden"></div>
  </div>

  <script src="bmi.js"></script>
</body>
</html>
```

```
const form   = document.getElementById('bmiForm');
const result = document.getElementById('result');

form.addEventListener('submit', function (event) {
    event.preventDefault();

    const heightCm = parseFloat(document.getElementById('height').value);
    const weightKg = parseFloat(document.getElementById('weight').value);

    if (isNaN(heightCm) || isNaN(weightKg) || heightCm <= 0 || weightKg <= 0) {
        result.textContent = 'Please enter valid positive numbers.';
        result.className = 'result error';
        return;
    }

    const heightM = heightCm / 100;
    const bmi     = weightKg / (heightM * heightM);

    let category, colour;

    if (bmi < 18.5) {
        category = 'Underweight'; colour = '#0369a1';
    } else if (bmi < 25) {
        category = 'Normal';      colour = '#15 803d';
    } else if (bmi < 30) {
        category = 'Overweight';  colour = '#b45 309';
    } else {
        category = 'Obese';       colour = '#b91c1c';
    }

    result.innerHTML =
        `<strong>Your BMI is ${bmi.toFixed(1)}</strong><br>` +
        `<span style="color:${colour}">Category: ${category}</span>`;
    result.className = 'result';
});
```

OUTPUT:

For a height of 172 cm and a weight of 68 kg the calculator reports BMI 23.0 and the category Normal. Entering a zero or blank value shows the validation message instead of a result.

RESULT:

Thus the BMI calculator was designed using HTML, CSS and JavaScript and the computed value and category were verified for several inputs.

RUBRICS :

Rubrics	Actual Marks	Awarded Marks
Understanding (R1)	4	
Timeline (R2)	4	
Design (R3)	4	
Development (R4)	4	
Innovation (R5)	4	
Total Marks	**20**	

Ex No: 03 22/07/2026

User registration form validation using JavaScript

AIM:

To design a user registration form and validate every field on the client side using JavaScript regular expressions before the form is submitted.

PROCEDURE:

1. Create register.html, register.css and validate.js.
2. Build the form with name, email, mobile, password, confirm password and gender fields.
3. Reserve an inline error span under each field ().
4. Attach an input listener to each field so the error clears as the user types.
5. Write a rule table mapping each field name to a validation function.
6. Validate the name length, the email pattern, the ten-digit mobile number, the password strength and the confirm-password match.
7. On submit, run every rule, show the first error and focus that field.
8. Call event.preventDefault() when any rule fails so nothing is posted.
9. Re-check the rules on the server as well, since client checks can be bypassed.
10. Test with a valid record, an invalid email, a nine-digit mobile and mismatched passwords.

JAVASCRIPT FEATURES USED:

Regular expressions, addEventListener, querySelectorAll, classList, textContent, focus, event.preventDefault, form dataset attributes.

CODE:

```
const rules = {
  full_name: v => v.trim().length >= 3
            || 'Enter your full name (min 3 characters)',
  email:    v => /^[^\s@]+@[^\s@]+\.[a-z]{2,}$/i.test(v.trim())
            || 'Enter a valid email address',
  phone:    v => /^[6-9]\d{9}$/.test(v.trim())
            || 'Enter a valid 10-digit mobile number',
  password: v => v.length >= 8
            || 'Password must be at least 8 characters',
  confirm:  v => v === document.getElementById('password').value
            || 'Passwords do not match'
};

document.querySelectorAll('form[data-validate]').forEach(function (form) {
  form.addEventListener('submit', function (event) {
    let firstBad = null;

    form.querySelectorAll('input, select, textarea').forEach(function (field) {
      const rule   = rules[field.name];
      const msgBox = form.querySelector(
        '[data-error-for="' + field.name + '"]');
      if (!rule) return;

      const result = rule(field.value);
      const ok     = result === true;
```

```
            field.classList.toggle('invalid', !ok);
            if (msgBox) {
                msgBox.textContent   = ok ? '' : result;
                msgBox.style.display = ok ? 'none' : 'block';
            }
            if (!ok && !firstBad) firstBad = field;
        });

        if (firstBad) {
            event.preventDefault();
            firstBad.focus();
        }
    });
});
```

SERVER-SIDE RE-CHECK:

```
if (mb_strlen($name) < 3) {
    $errors['full_name'] = 'Enter your full name (minimum 3 characters).';
}
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors['email'] = 'Enter a valid email address.';
}
if (!preg_match('/^[6-9]\d{9}$/', $phone)) {
    $errors['phone'] = 'Enter a valid 10-digit Indian mobile number.';
}
if (strlen($pass) < 8) {
    $errors['password'] = 'Password must be at least 8 characters.';
} elseif ($pass !== $confirm) {
    $errors['confirm'] = 'Passwords do not match.';
}
```

OUTPUT:

A valid submission proceeds to the server. An invalid email, a nine-digit mobile number or a mismatched password prefixes the field with a red message and the form does not submit.

RESULT:

Thus the user registration form was designed and validated on the client side with JavaScript and re-validated on the server with PHP.

RUBRICS :

Rubrics	Actual Marks	Awarded Marks
Understanding (R1)	4	
Timeline (R2)	4	
Design (R3)	4	
Development (R4)	4	
Innovation (R5)	4	
Total Marks	**20**	

Ex No: 04 29/07/2026

Implementing web applications with API calls using JavaScript

AIM:

To build a web page that fetches data from a remote REST API with the Fetch API, renders the response dynamically and handles loading and error states.

PROCEDURE:

1. Create api.html, api.css and api.js.
2. Add a search box, a category selector and a results container.
3. Use fetch() with the async/await syntax to call the JSON endpoint.
4. Show a loading indicator before the request and hide it on completion.
5. Check response.ok and throw an Error carrying the HTTP status on failure.
6. Parse the body with response.json() and render one card per record.
7. Wrap the call in try/catch/finally so network failures show a friendly message.
8. Debounce the search input so the API is not called on every keystroke.
9. Use AbortController to cancel an in-flight request when a new one starts.
10. Verify behaviour with a valid query, an empty query and the network switched off.

JAVASCRIPT FEATURES USED:

fetch, async/await, Promises, response.json, AbortController, try/catch/finally, template literals, Array.map, debouncing with setTimeout.

CODE:

```
const list    = document.getElementById('list');
const status  = document.getElementById('status');
const search  = document.getElementById('search');
let   controller = null;

async function loadProperties(query = '') {
   // cancel any request still in flight
   if (controller) controller.abort();
   controller = new AbortController();

   status.textContent = 'Loading…';

   try {
      const res = await fetch(
         'api/properties.php?q=' + encodeURIComponent(query),
         { signal: controller.signal }
      );

      if (!res.ok) {
         throw new Error('Request failed: ' + res.status);
      }

      const data = await res.json();

      if (!data.length) {
         list.innerHTML = '<p class="empty">No properties matched.</p>';
         status.textContent = '';
```

```
            return;
        }

        list.innerHTML = data.map(p => `
            <article class="card">
                <h3>${p.title}</h3>
                <p>${p.locality}, ${p.city}</p>
                <p class="rent">Rs. ${Number(p.rent_amount).toLocaleString('en-IN')}/mo</p>
            </article>
        `).join('');

        status.textContent = data.length + ' properties loaded';
    } catch (err) {
        if (err.name === 'AbortError') return;
        status.textContent = 'Could not load properties. ' + err.message;
    }
}

let timer;
search.addEventListener('input', function () {
    clearTimeout(timer);
    timer = setTimeout(() => loadProperties(this.value), 300);
});

loadProperties();
```

JSON ENDPOINT:

```
<?php
// api/properties.php - returns JSON instead of HTML
require_once __DIR__ . '/../includes/functions.php';
header('Content-Type: application/json; charset=utf-8');

$q = trim($_GET['q'] ?? '');

$stmt = db()->prepare(
    'SELECT property_id, title, city, locality, rent_amount, bhk
       FROM properties
      WHERE status = "available" AND title LIKE ?
      ORDER BY created_at DESC'
);
$stmt->execute(['%' . $q . '%']);

echo json_encode($stmt->fetchAll(), JSON_UNESCAPED_UNICODE);
```

OUTPUT:

The page lists properties as cards. Typing in the search box refreshes the list after a 300 ms debounce. Stopping the server shows the error message instead of an empty page.

RESULT:

Thus a web application consuming a REST endpoint with the Fetch API was implemented and the success, empty and error states were verified.

RUBRICS :

Rubrics	Actual Marks	Awarded Marks
Understanding (R1)	4	
Timeline (R2)	4	
Design (R3)	4	
Development (R4)	4	
Innovation (R5)	4	

Total Marks	**20**	

Ex No: 05 05/08/2026

Event registration form using PHP and MySQL

AIM:

To design a database-driven event registration form using HTML, PHP and MySQL that stores participant details, rejects duplicate emails and lists all registered participants.

PROCEDURE:

1. Start Apache and MySQL from the XAMPP control panel.
2. Create the database event_management and the table event_registrations.
3. Create db.php to open a MySQLi connection and set the charset to utf8mb4.
4. Design index.php with the participant name, email, mobile, institution, event name, category, number of participants and registration date.
5. Submit the form to register.php using the POST method.
6. Validate the input on the server: required fields, email format, mobile pattern.
7. Check for an existing row with the same email before inserting.
8. Insert the record with a prepared statement.
9. Show a confirmation message, or the validation errors, on the same page.
10. List every registration in participants.php with a SELECT query.
11. Test with valid data, a bad email, an empty field and a duplicate email.

DATABASE:

```
CREATE DATABASE IF NOT EXISTS event_management;
USE event_management;

CREATE TABLE IF NOT EXISTS event_registrations (
    registration_id   INT AUTO_INCREMENT PRIMARY KEY,
    participant_name  VARCHAR(100) NOT NULL,
    email             VARCHAR(150) NOT NULL UNIQUE,
    mobile            VARCHAR(15)  NOT NULL,
    institution       VARCHAR(150) NOT NULL,
    event_name        VARCHAR(150) NOT NULL,
    category          VARCHAR(50)  NOT NULL,
    participants      INT          NOT NULL,
    registration_date DATE         NOT NULL
);
```

DB.PHP:

```
<?php
$host     = "localhost";
$user     = "root";
$password = "YOUR_MYSQL_PASSWORD";
$database = "event_management";

$conn = new mysqli($host, $user, $password, $database);

if ($conn->connect_error) {
    error_log($conn->connect_error);
    die("Database connection failed.");
}
```

```
$conn->set_charset("utf8mb4");
?>
```

REGISTER.PHP:

```
<?php
require_once 'db.php';

$errors = [];
$done   = false;

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = trim($_POST['participant_name'] ?? '');
    $mail = trim($_POST['email'] ?? '');
    $mob  = trim($_POST['mobile'] ?? '');
    $inst = trim($_POST['institution'] ?? '');
    $evnt = trim($_POST['event_name'] ?? '');
    $cat  = $_POST['category'] ?? '';
    $pax  = (int) ($_POST['participants'] ?? 0);
    $date = $_POST['registration_date'] ?? '';

    if ($name === '')                    $errors[] = 'Name is required.';
    if (!filter_var($mail, FILTER_VALIDATE_EMAIL)) $errors[] = 'Invalid email address.';
    if (!preg_match('/^[6-9]\d{9}$/', $mob))      $errors[] = 'Invalid mobile number.';
    if ($pax < 1)                        $errors[] = 'Participants must be at least 1.';
    if ($date === '')                    $errors[] = 'Registration date is required.';

    if (!$errors) {
        $check = $conn->prepare('SELECT registration_id FROM event_registrations WHERE email = ?');
        $check->bind_param('s', $mail);
        $check->execute();
        if ($check->get_result()->fetch()) {
            $errors[] = 'This email is already registered.';
        }
    }

    if (!$errors) {
        $sql = 'INSERT INTO event_registrations
                (participant_name, email, mobile, institution,
                 event_name, category, participants, registration_date)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?)';
        $stmt = $conn->prepare($sql);
        $stmt->bind_param('ssssssis', $name, $mail, $mob, $inst,
                          $evnt, $cat, $pax, $date);
        $stmt->execute();
        $done = true;
    }
}
?>
```

OUTPUT:

A valid registration shows a confirmation panel and the record appears in participants.php. A duplicate email is refused with the message "This email is already registered."

RESULT:

Thus the event registration form was developed using HTML, PHP and MySQL and duplicate registration was prevented with a unique email constraint.

RUBRICS :

Rubrics	Actual Marks	Awarded Marks
Understanding (R1)	4	

Timeline (R2)	4	
Design (R3)	4	
Development (R4)	4	
Innovation (R5)	4	
Total Marks	**20**	

Ex No: 06 19/08/2026

Web application for food delivery system using PHP and MySQL

AIM:

To design a food delivery web application using PHP and MySQL where a user registers, logs in, browses the menu, places an order and receives an order confirmation, and an administrator manages the menu and order status.

PROCEDURE:

1. Create the project folder food-delivery inside htdocs and start Apache and MySQL.
2. Create the database food_delivery with the tables users, food_items, orders and order_items.
3. Write db.php to connect PHP to MySQL over MySQLi.
4. Build register.php and login.php with password_hash() and password_verify().
5. Build menu.php to read food_items and render one card per item.
6. Let the user set a quantity and add items to a session cart.
7. Collect the customer name, mobile number and delivery address at checkout.
8. Compute the total, insert the order and its line items in a transaction.
9. Show an order confirmation with the order id, items, total and status.
10. Build admin.php to add or remove items and advance the order status.
11. Test registration, login, ordering and the admin operations.

DATABASE:

```
CREATE DATABASE IF NOT EXISTS food_delivery;
USE food_delivery;

CREATE TABLE users (
    user_id  INT AUTO_INCREMENT PRIMARY KEY,
    name     VARCHAR(100) NOT NULL,
    email    VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    mobile   VARCHAR(15),
    address  TEXT
);

CREATE TABLE food_items (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    name    VARCHAR(100) NOT NULL,
    price   DECIMAL(8,2) NOT NULL,
    category VARCHAR(50),
    available TINYINT(1) DEFAULT 1
);

CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id  INT NOT NULL,
    total    DECIMAL(10,2) NOT NULL,
    address  TEXT NOT NULL,
    status   ENUM('placed','preparing','delivered','cancelled') DEFAULT 'placed',
    ordered_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

```
CREATE TABLE order_items (
    item_id  INT NOT NULL,
    order_id INT NOT NULL,
    qty      INT NOT NULL,
    price    DECIMAL(8,2) NOT NULL,
    PRIMARY KEY (item_id, order_id),
    FOREIGN KEY (item_id)  REFERENCES food_items(item_id),
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE
);
```

CHECKOUT.PHP (TRANSACTION):

```
<?php
session_start();
require_once 'db.php';

if (empty($_SESSION['cart'])) {
    die('Your cart is empty.');
}

$address = trim($_POST['address'] ?? '');
$total   = 0.0;

foreach ($_SESSION['cart'] as $itemId => $qty) {
    $stmt = $conn->prepare('SELECT price FROM food_items WHERE item_id = ?');
    $stmt->bind_param('i', $itemId);
    $stmt->execute();
    $row = $stmt->get_result()->fetch_assoc();
    $total += $row['price'] * $qty;
}

$conn->begin_transaction();

try {
    $ins = $conn->prepare(
        'INSERT INTO orders (user_id, total, address) VALUES (?, ?, ?)');
    $ins->bind_param('ids', $_SESSION['user_id'], $total, $address);
    $ins->execute();
    $orderId = $conn->insert_id;

    $line = $conn->prepare(
        'INSERT INTO order_items (order_id, item_id, qty, price)
         VALUES (?, ?, ?, ?)');

    foreach ($_SESSION['cart'] as $itemId => $qty) {
        $price = /* current price from food_items */ 0;
        $line->bind_param('iiid', $orderId, $itemId, $qty, $price);
        $line->execute();
    }

    $conn->commit();
    $_SESSION['cart'] = [];
    header('Location: confirmation.php?order=' . $orderId);
} catch (Throwable $e) {
    $conn->rollback();
    die('Order could not be placed.');
}
```

OUTPUT:

The menu lists the available dishes, the cart totals the selected quantities and checkout writes the order and its lines in one transaction. The confirmation page shows the order id and status, which the administrator can advance from Placed to Preparing to Delivered.

RESULT:

Thus the food delivery web application was developed using PHP and MySQL with session-based cart handling and transactional order insertion.

RUBRICS :

Rubrics	Actual Marks	Awarded Marks
Understanding (R1)	4	
Timeline (R2)	4	
Design (R3)	4	
Development (R4)	4	
Innovation (R5)	4	
Total Marks	**20**	

Ex No: 07 02/09/2026

Web application for leave management using Servlet and MySQL

AIM:

To design a leave management web application using Jakarta Servlets, JSP, JDBC and MySQL in which an employee applies for leave, a manager approves or rejects it, and the leave balance is maintained.

PROCEDURE:

1. Install JDK 17, Apache Maven, MySQL and Apache Tomcat 10.1.
2. Create the database leave_management with employee and leave_request tables.
3. Create a Maven web project and add the Jakarta Servlet and MySQL connector dependencies.
4. Write DB.java to return a JDBC connection from DriverManager.
5. Build login.jsp and a LoginServlet that stores the employee id in the session.
6. Build apply.jsp and an ApplyLeaveServlet that validates and inserts a leave request.
7. Add an AuthFilter that blocks protected URLs when no session exists.
8. Build ListLeaveServlet to show the signed-in employee's requests.
9. Build ApproveLeaveServlet so a manager can approve or reject a pending request.
10. Deduct the approved days from the leave balance inside a transaction.
11. Package with mvn clean package, deploy the WAR on Tomcat and test every workflow.

POM.XML:

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.leave</groupId>
  <artifactId>leave-management</artifactId>
  <version>1.0</version>
  <packaging>war</packaging>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
  </properties>

  <dependencies>
    <dependency>
      <groupId>jakarta.servlet</groupId>
      <artifactId>jakarta.servlet-api</artifactId>
      <version>6.0.0</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>9.4.0</version>
    </dependency>
  </dependencies>

  <build>
    <finalName>leave-management</finalName>
  </build>
</project>
```

DATABASE - LEAVE_MANAGEMENT.SQL:

```
CREATE DATABASE leave_management;
USE leave_management;

CREATE TABLE employees (
   emp_id   INT PRIMARY KEY AUTO_INCREMENT,
   name     VARCHAR(80)  NOT NULL,
   email    VARCHAR(120) NOT NULL UNIQUE,
   password VARCHAR(100) NOT NULL,
   role     ENUM('employee','manager') DEFAULT 'employee',
   balance  INT DEFAULT 20
);

CREATE TABLE leave_requests (
   request_id INT PRIMARY KEY AUTO_INCREMENT,
   emp_id     INT NOT NULL,
   leave_type ENUM('Casual','Sick','Earned') NOT NULL,
   from_date  DATE NOT NULL,
   to_date    DATE NOT NULL,
   days       INT  NOT NULL,
   reason     VARCHAR(255),
   status     ENUM('pending','approved','rejected') DEFAULT 'pending',
   applied_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   FOREIGN KEY (emp_id) REFERENCES employees(emp_id)
);

INSERT INTO employees (name, email, password, role, balance)
VALUES ('Pavi', 'pavi@org.in', '1234', 'employee', 20),
      ('Ravi', 'ravi@org.in', '1234', 'manager', 20);
```

APPLYLEAVESERVLET.JAVA:

```
package com.leave;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;
import java.io.*;
import java.sql.*;
import java.time.*;
import java.time.temporal.ChronoUnit;

@WebServlet("/apply")
public class ApplyLeaveServlet extends HttpServlet {

   protected void doPost(HttpServletRequest req,
                         HttpServletResponse res)
         throws ServletException, IOException {

      HttpSession session = req.getSession(false);
      if (session == null || session.getAttribute("empId") == null) {
         res.sendRedirect("login.jsp");
         return;
      }

      int empId      = (Integer) session.getAttribute("empId");
      String type    = req.getParameter("leave_type");
      LocalDate from = LocalDate.parse(req.getParameter("from_date"));
      LocalDate to   = LocalDate.parse(req.getParameter("to_date"));
      String reason  = req.getParameter("reason");

      if (to.isBefore(from)) {
         res.getWriter().println("End date must be after the start date.");
         return;
      }

      long days = ChronoUnit.DAYS.between(from, to) + 1;
```

```
        try (Connection con = DB.getConnection();
             PreparedStatement ps = con.prepareStatement(
                 "INSERT INTO leave_requests " +
                 "(emp_id, leave_type, from_date, to_date, days, reason) " +
                 "VALUES (?,?,?,?,?,?)")) {

            ps.setInt(1, empId);
            ps.setString(2, type);
            ps.setDate(3, Date.valueOf(from));
            ps.setDate(4, Date.valueOf(to));
            ps.setLong(5, days);
            ps.setString(6, reason);
            ps.executeUpdate();

            res.sendRedirect("list");

        } catch (Exception e) {
            throw new ServletException(e);
        }
    }
}
```

APPROVELEAVESERVLET.JAVA:

```
package com.leave;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;
import java.io.*;
import java.sql.*;

@WebServlet("/approve")
public class ApproveLeaveServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req,
                          HttpServletResponse res)
            throws ServletException, IOException {

        int requestId = Integer.parseInt(req.getParameter("request_id"));
        String action = req.getParameter("action");   // approve | reject

        try (Connection con = DB.getConnection()) {
            con.setAutoCommit(false);

            try {
                PreparedStatement sel = con.prepareStatement(
                    "SELECT emp_id, days FROM leave_requests " +
                    "WHERE request_id = ? AND status = 'pending'");
                sel.setInt(1, requestId);
                ResultSet rs = sel.executeQuery();

                if (rs.next()) {
                    int empId = rs.getInt("emp_id");
                    int days  = rs.getInt("days");

                    con.prepareStatement(
                        "UPDATE leave_requests SET status = '" +
                        ("approve".equals(action) ? "approved" : "rejected") +
                        "' WHERE request_id = " + requestId
                    ).executeUpdate();

                    if ("approve".equals(action)) {
                        PreparedStatement bal = con.prepareStatement(
                            "UPDATE employees SET balance = balance - ? " +
                            "WHERE emp_id = ?");
                        bal.setInt(1, days);
                        bal.setInt(2, empId);
                        bal.executeUpdate();
```

```
                }
            }

            con.commit();
        } catch (Exception e) {
            con.rollback();
            throw e;
        }

        res.sendRedirect("list");

    } catch (Exception e) {
        throw new ServletException(e);
    }
  }
}
```

OUTPUT:

The employee logs in, applies for three days of casual leave and the request appears as Pending. The manager approves it, the status becomes Approved and the employee's balance drops from 20 to 17.

RESULT:

Thus the leave management web application was developed using Jakarta Servlets, JSP, JDBC and MySQL and the apply and approve workflows were verified on Tomcat.

RUBRICS :

Rubrics	Actual Marks	Awarded Marks
Understanding (R1)	4	
Timeline (R2)	4	
Design (R3)	4	
Development (R4)	4	
Innovation (R5)	4	
Total Marks	**20**	

Ex No: 08 16/09/2026

Web application for Online Shopping using Servlet and MySQL

AIM:

To design and develop a simple online shopping web application using Java, Jakarta Servlets, JSP, JDBC and MySQL that allows a user to log in, view products, add and remove products in a cart and compute the total price.

PROCEDURE:

1. Install JDK 17, Apache Maven, MySQL and Apache Tomcat 10.1.
2. Create a MySQL database named online_shopping.
3. Create the users and products tables and insert sample records.
4. Create a Maven web application and configure the dependencies in pom.xml.
5. Develop a login page using HTML and JSP.
6. Create a Login Servlet to validate credentials through JDBC.
7. Develop a product dashboard that lists products from MySQL.
8. Implement the shopping cart using an HTTP session.
9. Create a cart page showing the selected products, quantities and total.
10. Apply simple CSS to improve the appearance.
11. Build with mvn clean package and deploy the WAR on Tomcat.
12. Test login, product display, add, remove and the total calculation.

DATABASE - ONLINE_SHOPPING.SQL:

```
CREATE DATABASE online_shopping;
USE online_shopping;

CREATE TABLE users (
    id       INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50),
    password VARCHAR(50)
);

CREATE TABLE products (
    id    INT PRIMARY KEY AUTO_INCREMENT,
    name  VARCHAR(100),
    price DOUBLE
);

INSERT INTO users(username, password) VALUES ('pavi', '1234');

INSERT INTO products(name, price) VALUES
('Laptop', 45 000),
('Headphones', 1500),
('Keyboard', 800),
('Mouse', 500);
```

LOGINSERVLET.JAVA:

```
package com.shop;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
```

```
import jakarta.servlet.annotation.*;
import java.io.*;
import java.sql.*;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req,
                          HttpServletResponse res)
            throws ServletException, IOException {

        String username = req.getParameter("username");
        String password = req.getParameter("password");

        try (Connection con = DB.getConnection();
             PreparedStatement ps = con.prepareStatement(
                "SELECT * FROM users WHERE username=? AND password=?")) {

            ps.setString(1, username);
            ps.setString(2, password);

            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                HttpSession session = req.getSession();
                session.setAttribute("user", username);
                res.sendRedirect("products");
            } else {
                res.getWriter().println("Invalid username or password");
            }

        } catch (Exception e) {
            throw new ServletException(e);
        }
    }
}
```

CARTSERVLET.JAVA:

```
package com.shop;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;
import java.io.*;
import java.sql.*;
import java.util.*;

@WebServlet("/cart")
public class CartServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req,
                          HttpServletResponse res)
            throws IOException {

        HttpSession session = req.getSession(false);

        if (session == null || session.getAttribute("user") == null) {
            res.sendRedirect("login.jsp");
            return;
        }

        Map<Integer, Integer> cart =
            (Map<Integer, Integer>) session.getAttribute("cart");

        if (cart == null) {
            cart = new HashMap<>();
            session.setAttribute("cart", cart);
        }
```

```
    int id = Integer.parseInt(req.getParameter("id"));
    String action = req.getParameter("action");

    if ("add".equals(action)) {
        cart.put(id, cart.getOrDefault(id, 0) + 1);
    }
    if ("remove".equals(action)) {
        cart.remove(id);
    }

    res.sendRedirect("cart");
}

protected void doGet(HttpServletRequest req,
                     HttpServletResponse res)
        throws ServletException, IOException {

    HttpSession session = req.getSession(false);

    if (session == null || session.getAttribute("user") == null) {
        res.sendRedirect("login.jsp");
        return;
    }

    Map<Integer, Integer> cart =
        (Map<Integer, Integer>) session.getAttribute("cart");

    if (cart == null) cart = new HashMap<>();

    StringBuilder rows = new StringBuilder();
    double total = 0;

    try (Connection con = DB.getConnection();
         PreparedStatement ps = con.prepareStatement(
             "SELECT * FROM products WHERE id=?")) {

        for (int id : cart.keySet()) {
            ps.setInt(1, id);

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    String name  = rs.getString("name");
                    double price = rs.getDouble("price");
                    int    qty   = cart.get(id);
                    double sub   = price * qty;

                    total += sub;

                    rows.append("<tr><td>").append(name)
                        .append("</td><td>Rs.").append(price)
                        .append("</td><td>").append(qty)
                        .append("</td><td>Rs.").append(sub)
                        .append("</td><td><form method='post' action='cart'>")
                        .append("<input type='hidden' name='id' value='")
                        .append(id)
                        .append("'><button name='action' value='remove'>")
                        .append("Remove</button></form></td></tr>");
                }
            }
        }

        req.setAttribute("rows", rows.toString());
        req.setAttribute("total", total);
        req.getRequestDispatcher("/cart.jsp").forward(req, res);

    } catch (Exception e) {
        throw new ServletException(e);
    }
```

```
    }
}
```

OUTPUT:

After login the product dashboard lists the four products. Adding a laptop and two headphones shows three rows in the cart with the total Rs.48,000. Removing the laptop recalculates the total to Rs.3,000.

RESULT:

Thus the online shopping web application was designed and developed using Java, Jakarta Servlets, JSP, JDBC and MySQL and the login, product display and cart operations were verified on Tomcat.

RUBRICS :

Rubrics	Actual Marks	Awarded Marks
Understanding (R1)	4	
Timeline (R2)	4	
Design (R3)	4	
Development (R4)	4	
Innovation (R5)	4	
Total Marks	**20**	

Ex No: 09 **02/11/2026**

Mini Project - RentEase: Rental Property Management System

AIM:

To design and develop a complete rental property management web application, "RentEase", using HTML, CSS, JavaScript, PHP and MySQL that lets property owners publish listings with images and lets tenants search those listings, submit rental applications, record monthly rent payments, raise maintenance complaints, and gives the owner a dashboard to manage the whole portfolio.

OBJECTIVES:

- Provide a searchable, filterable catalogue of rental properties with image galleries.
- Let a tenant apply for a property online and let the owner approve or reject the application.
- Maintain a month-by-month rent payment record against a transaction reference.
- Track maintenance complaints from Open through In-Progress to Resolved.
- Give the owner a single dashboard for listings, applications, rent collected and complaints.
- Enforce role-based access, CSRF protection and prepared statements throughout.

SCOPE OF THE SYSTEM:

Module	Actors	Functions
Authentication	Owner, Tenant	Register, log in, log out, role-based redirection
Property listing	Owner	Add listing, upload cover image, toggle availability
Search & filter	Tenant, Guest	Keyword, city, BHK, type, furnishing, budget, sort, paging
Rental application	Tenant, Owner	Apply with occupation and family size; approve or reject
Rent payment	Tenant, Owner	Record monthly payment, transaction reference, receipt list
Maintenance	Tenant, Owner	Raise complaint, set priority, update status, owner note
Owner dashboard	Owner	Portfolio stats, rent chart, application queue, complaint queue

TECHNOLOGY STACK:

Layer	Technology	Used for
Markup	HTML5	Semantic header, nav, section, article and footer; forms and tables
Styling	CSS3	Custom properties, flexbox, grid, media queries, hover states
Behaviour	JavaScript (ES6)	Field validation, live filtering, gallery, rent preview
Server	PHP 8.4	Session handling, PDO queries, password hashing, file upload
Database	MySQL 9	Six InnoDB tables, foreign keys, ENUM and unique constraints

SYSTEM ARCHITECTURE:

RentEase follows a three-tier architecture. The browser renders HTML built by PHP and styles it with one stylesheet; the PHP layer authenticates the user, validates the request and talks to MySQL only through PDO prepared statements; the data layer stores six related tables in InnoDB.

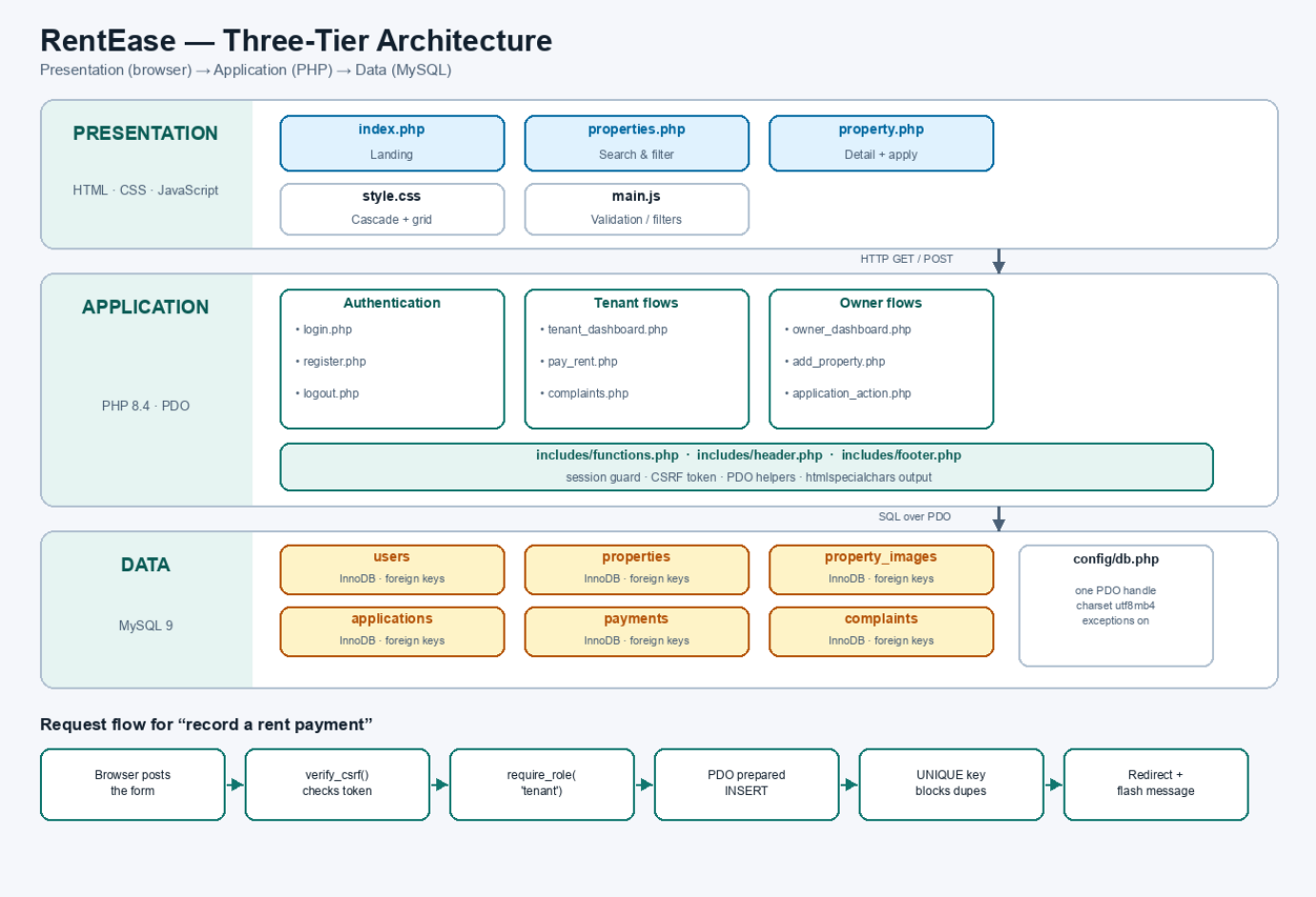


Fig. 9.1 Three-tier architecture of RentEase and the request flow for a rent payment

ENTITY RELATIONSHIP DIAGRAM:

A single users table holds both owners and tenants, distinguished by the role column. One owner has many properties; one property has many images, applications, payments and complaints; a tenant links to properties through applications and payments.

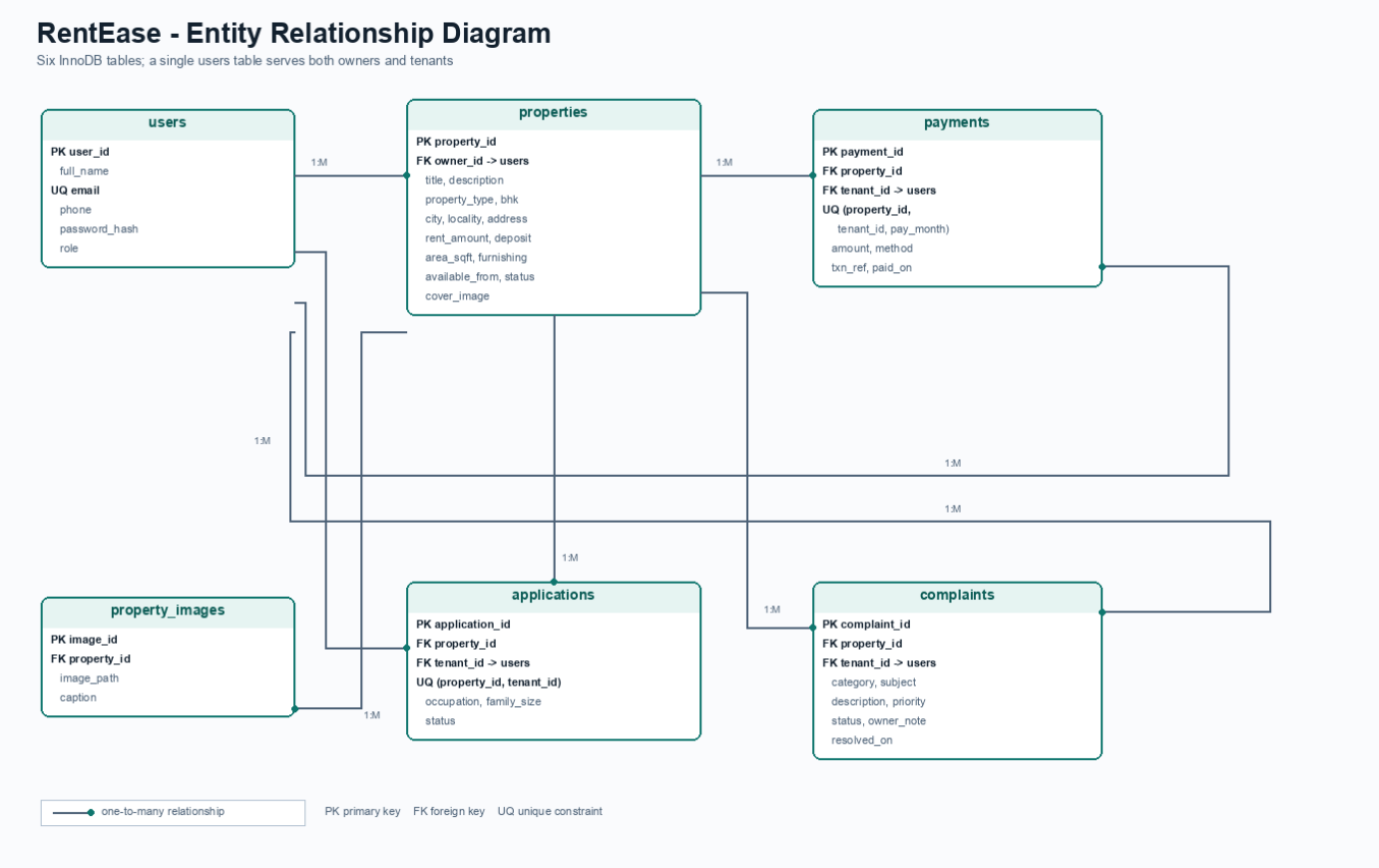


Fig. 9.2 Entity relationship diagram of the rentease database

PROCEDURE:

1. Install PHP 8.4, MySQL 9 and a web browser; XAMPP may be used instead of a native install.
2. Create the project folder RentEase with the subfolders config, includes, assets, database, scripts and screenshots.
3. Create the database rentease and the six tables users, properties, property_images, applications, payments and complaints.
4. Write config/db.php to open one shared PDO connection with exceptions enabled and utf8mb4 as the charset.
5. Write includes/functions.php with the session bootstrap, require_login(), require_role(), the CSRF helpers and the e() output escaper.
6. Build includes/header.php and includes/footer.php so every page shares the same navigation and footer.
7. Design index.php with a hero banner, a quick search form and the featured listings.
8. Design properties.php with keyword, city, BHK, type, furnishing and budget filters built as a dynamically assembled WHERE clause with bound parameters.
9. Design property.php with an image gallery, a specification sheet and the rental application form.
10. Implement register.php and login.php with password_hash(), password_verify() and session_regenerate_id().
11. Implement tenant_dashboard.php, pay_rent.php and complaints.php for the tenant side.

12. Implement owner_dashboard.php, add_property.php, application_action.php, complaint_action.php and property_action.php for the owner side.
13. Write assets/css/style.css and assets/js/main.js for styling and client-side behaviour.
14. Populate the tables with representative demo data from database/rentease.sql.
15. Start the server with php -S 127.0.0.1:8088, exercise every workflow and record the screenshots for this report.

DATABASE - RENTEASE.SQL:

```
DROP DATABASE IF EXISTS rentease;
CREATE DATABASE rentease
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;
USE rentease;

CREATE TABLE users (
  user_id       INT AUTO_INCREMENT PRIMARY KEY,
  full_name     VARCHAR(80)  NOT NULL,
  email         VARCHAR(120) NOT NULL UNIQUE,
  phone         VARCHAR(15)  NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role          ENUM('owner','tenant') NOT NULL DEFAULT 'tenant',
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE properties (
  property_id   INT AUTO_INCREMENT PRIMARY KEY,
  owner_id      INT NOT NULL,
  title         VARCHAR(120) NOT NULL,
  description   TEXT,
  property_type ENUM('Apartment','Independent House','Villa',
                 'PG / Hostel','Studio') NOT NULL,
  bhk           TINYINT NOT NULL,
  city          VARCHAR(60) NOT NULL,
  locality      VARCHAR(80) NOT NULL,
  address       VARCHAR(200) NOT NULL,
  rent_amount   DECIMAL(10,2) NOT NULL,
  deposit       DECIMAL(10,2) NOT NULL DEFAULT 0,
  area_sqft     INT NOT NULL,
  furnishing    ENUM('Unfurnished','Semi-Furnished','Fully-Furnished') NOT NULL,
  available_from DATE NOT NULL,
  status        ENUM('available','occupied','inactive') DEFAULT 'available',
  cover_image   VARCHAR(200),
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_prop_owner FOREIGN KEY (owner_id)
    REFERENCES users(user_id) ON DELETE CASCADE,
  INDEX idx_city (city), INDEX idx_rent (rent_amount),
  INDEX idx_bhk (bhk),   INDEX idx_status (status)
) ENGINE=InnoDB;

CREATE TABLE property_images (
  image_id    INT AUTO_INCREMENT PRIMARY KEY,
  property_id INT NOT NULL,
  image_path  VARCHAR(200) NOT NULL,
  caption     VARCHAR(120),
  CONSTRAINT fk_img_prop FOREIGN KEY (property_id)
    REFERENCES properties(property_id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE applications (
  application_id INT AUTO_INCREMENT PRIMARY KEY,
  property_id    INT NOT NULL,
  tenant_id      INT NOT NULL,
  message        TEXT,
  occupation     VARCHAR(60),
```

```
    family_size    TINYINT DEFAULT 1,
    applied_on     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status         ENUM('pending','approved','rejected') DEFAULT 'pending',
    UNIQUE KEY uq_app (property_id, tenant_id),
    CONSTRAINT fk_app_prop   FOREIGN KEY (property_id)
        REFERENCES properties(property_id) ON DELETE CASCADE,
    CONSTRAINT fk_app_tenant FOREIGN KEY (tenant_id)
        REFERENCES users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE payments (
    payment_id  INT AUTO_INCREMENT PRIMARY KEY,
    property_id INT NOT NULL,
    tenant_id   INT NOT NULL,
    amount      DECIMAL(10,2) NOT NULL,
    pay_month   DATE NOT NULL,
    method      ENUM('UPI','Net Banking','Card','Cash') NOT NULL,
    txn_ref     VARCHAR(40) NOT NULL,
    paid_on     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_pay (property_id, tenant_id, pay_month),
    CONSTRAINT fk_pay_prop   FOREIGN KEY (property_id)
        REFERENCES properties(property_id) ON DELETE CASCADE,
    CONSTRAINT fk_pay_tenant FOREIGN KEY (tenant_id)
        REFERENCES users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE complaints (
    complaint_id INT AUTO_INCREMENT PRIMARY KEY,
    property_id  INT NOT NULL,
    tenant_id    INT NOT NULL,
    category     ENUM('Plumbing','Electrical','Carpentry',
                      'Appliance','Cleaning','Other') NOT NULL,
    subject      VARCHAR(120) NOT NULL,
    description  TEXT NOT NULL,
    priority     ENUM('Low','Medium','High') DEFAULT 'Medium',
    status       ENUM('open','in-progress','resolved') DEFAULT 'open',
    owner_note   VARCHAR(255),
    raised_on    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    resolved_on  DATETIME NULL,
    CONSTRAINT fk_cmp_prop   FOREIGN KEY (property_id)
        REFERENCES properties(property_id) ON DELETE CASCADE,
    CONSTRAINT fk_cmp_tenant FOREIGN KEY (tenant_id)
        REFERENCES users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

CONFIG/DB.PHP:

One PDO handle is shared by every page. Exceptions are enabled so a failed query surfaces immediately instead of failing silently.

```
<?php
declare(strict_types=1);

const DB_HOST = '127.0.0.1';
const DB_PORT = 3306;      // 3307 in the demo environment
const DB_NAME = 'rentease';
const DB_USER = 'root';
const DB_PASS = '';        // set your local MySQL password

function db(): PDO
{
    static $pdo = null;
    if ($pdo instanceof PDO) {
        return $pdo;
    }

    $dsn = sprintf('mysql:host=%s;port=%d;dbname=%s;charset=utf8mb4',
        DB_HOST, DB_PORT, DB_NAME);
```

```
    try {
        $pdo = new PDO($dsn, DB_USER, DB_PASS, [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES   => false,
        ]);
    } catch (PDOException $e) {
        http_response_code(500);
        exit('Database connection failed.');
    }

    return $pdo;
}
```

INCLUDES/FUNCTIONS.PHP:

```
<?php
require_once __DIR__ . '/../config/db.php';

if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

function current_user(): ?array
{
    return $_SESSION['user'] ?? null;
}

function is_owner(): bool
{
    return (current_user()['role'] ?? '') === 'owner';
}

function require_login(): void
{
    if (!current_user()) {
        header('Location: login.php?next=' .
               urlencode($_SERVER['REQUEST_URI'] ?? 'index.php'));
        exit;
    }
}

function require_role(string $role): void
{
    require_login();
    if ((current_user()['role'] ?? '') !== $role) {
        http_response_code(403);
        exit('403 - forbidden');
    }
}

function csrf_token(): string
{
    if (empty($_SESSION['csrf'])) {
        $_SESSION['csrf'] = bin2hex(random_bytes(16));
    }
    return $_SESSION['csrf'];
}

function csrf_field(): string
{
    return '<input type="hidden" name="csrf" value="' . csrf_token() . '">';
}

function verify_csrf(): void
{
    $sent = $_POST['csrf'] ?? '';
    if (!is_string($sent) || !hash_equals($_SESSION['csrf'] ?? '', $sent)) {
```

```
        http_response_code(419);
        exit('Session expired. Please reload the page.');
    }
}

function e(?string $value): string
{
    return htmlspecialchars((string) $value, ENT_QUOTES, 'UTF-8');
}

function money(float|string $amount): string
{
    return 'Rs.' . number_format((float) $amount, 0, '.', ',');
}
```

LOGIN.PHP - AUTHENTICATION:

```
<?php
require_once __DIR__ . '/includes/functions.php';

if (is_logged_in()) {
    redirect(is_owner() ? 'owner_dashboard.php' : 'tenant_dashboard.php');
}

$error = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    verify_csrf();

    $email = trim($_POST['email'] ?? '');
    $pass  = $_POST['password'] ?? '';

    $stmt = db()->prepare('SELECT * FROM users WHERE email = ? LIMIT 1');
    $stmt->execute([$email]);
    $user = $stmt->fetch();

    if ($user && password_verify($pass, $user['password_hash'])) {
        session_regenerate_id(true);          // stop session fixation
        $_SESSION['user'] = [
            'user_id'   => (int) $user['user_id'],
            'full_name' => $user['full_name'],
            'role'      => $user['role'],
        ];
        flash('Logged in as ' . $user['full_name'] . '.');
        redirect($user['role'] === 'owner'
            ? 'owner_dashboard.php'
            : 'tenant_dashboard.php');
    }

    $error = 'Invalid email or password. Please try again.';
}

$pageTitle = 'Log in';
require __DIR__ . '/includes/header.php';
?>
```

PROPERTIES.PHP - SEARCH AND FILTER:

Filters are appended to a WHERE array only when the visitor supplied a value, and every value is bound. The ORDER BY clause is chosen from a fixed match() expression, so no user string ever reaches the SQL text.

```
<?php
require_once __DIR__ . '/includes/functions.php';

const PER_PAGE = 6;

$q       = trim($_GET['q'] ?? '');
$city    = trim($_GET['city'] ?? '');
$bhk     = trim($_GET['bhk'] ?? '');
$type    = trim($_GET['type'] ?? '');
$maxRent = trim($_GET['max_rent'] ?? '');
$furnish = trim($_GET['furnishing'] ?? '');
$sort    = $_GET['sort'] ?? 'newest';
$page    = max(1, (int) ($_GET['page'] ?? 1));

$where  = ['p.status <> "inactive"'];
$params = [];

if ($q !== '') {
    $where[] = '(p.title LIKE ? OR p.locality LIKE ? OR p.city LIKE ? '
             . 'OR p.address LIKE ?)';
```

```
    $like = '%' . $q . '%';
    array_push($params, $like, $like, $like, $like);
}
if ($city !== '') {
    $where[]  = 'p.city = ?';
    $params[] = $city;
}
if ($bhk !== '') {
    $where[]  = 'p.bhk = ?';
    $params[] = (int) $bhk;
}
if ($type !== '') {
    $where[]  = 'p.property_type = ?';
    $params[] = $type;
}
if ($maxRent !== '') {
    $where[]  = 'p.rent_amount <= ?';
    $params[] = (float) $maxRent;
}
if ($furnish !== '') {
    $where[]  = 'p.furnishing = ?';
    $params[] = $furnish;
}

$orderBy = match ($sort) {
    'rent_asc'  => 'p.rent_amount ASC',
    'rent_desc' => 'p.rent_amount DESC',
    'area_desc' => 'p.area_sqft DESC',
    default     => 'p.created_at DESC',
};

$sqlWhere = 'WHERE ' . implode(' AND ', $where);

$cnt = db()->prepare("SELECT COUNT(*) FROM properties p $sqlWhere");
$cnt->execute($params);
$total  = (int) $cnt->fetchColumn();
$pages  = max(1, (int) ceil($total / PER_PAGE));
$page   = min($page, $pages);
$offset = ($page - 1) * PER_PAGE;

$stmt = db()->prepare(
    "SELECT p.*, u.full_name AS owner_name
       FROM properties p
       JOIN users u ON u.user_id = p.owner_id
      $sqlWhere
      ORDER BY $orderBy
      LIMIT " . PER_PAGE . " OFFSET $offset"
);
$stmt->execute($params);
$rows = $stmt->fetchAll();
?>
```

PROPERTY.PHP - RENTAL APPLICATION:

```
<?php
$id = (int) ($_GET['id'] ?? 0);

$stmt = db()->prepare(
    'SELECT p.*, u.full_name AS owner_name
       FROM properties p
       JOIN users u ON u.user_id = p.owner_id
      WHERE p.property_id = ?'
);
$stmt->execute([$id]);
$p = $stmt->fetch();

if (!$p) {
    http_response_code(404);
    exit('Listing not found');
```

```
}

/* show the current tenant's status, if they already applied */
$myApplication = null;
if (is_tenant()) {
    $s = db()->prepare('SELECT status, applied_on FROM applications
                        WHERE property_id = ? AND tenant_id = ?');
    $s->execute([$id, current_user()['user_id']]);
    $myApplication = $s->fetch() ?: null;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    require_login();

    if (!is_tenant()) {
        $error = 'Only tenant accounts can apply for a property.';
    } else {
        verify_csrf();

        $message    = trim($_POST['message'] ?? '');
        $occupation = trim($_POST['occupation'] ?? '');
        $family     = max(1, (int) ($_POST['family_size'] ?? 1));

        if ($myApplication) {
            $error = 'You have already applied for this property.';
        } elseif (mb_strlen($message) < 15) {
            $error = 'Please write at least 15 characters about yourself.';
        } else {
            $ins = db()->prepare(
                'INSERT INTO applications
                   (property_id, tenant_id, message, occupation, family_size)
                 VALUES (?, ?, ?, ?, ?)'
            );
            $ins->execute([$id, current_user()['user_id'],
                           $message, $occupation, $family]);

            flash('Application submitted. The owner will review it shortly.');
            redirect('property.php?id=' . $id);
        }
    }
}
?>
```

PAY_RENT.PHP - RENT PAYMENT RECORD:

The amount is taken from the properties row rather than from the form, so a tenant cannot alter it, and the unique key on (property_id, tenant_id, pay_month) makes a second payment for the same month fail at the database.

```
<?php
require_once __DIR__ . '/includes/functions.php';
require_role('tenant');

$uid = (int) current_user()['user_id'];

/* only approved rentals can receive a payment */
$rentals = db()->prepare(
    'SELECT p.property_id, p.title, p.rent_amount
       FROM applications a
       JOIN properties p ON p.property_id = a.property_id
      WHERE a.tenant_id = ? AND a.status = "approved"'
);
$rentals->execute([$uid]);
$rentals = $rentals->fetchAll();

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    verify_csrf();
```

```
    $propId = (int) ($_POST['property_id'] ?? 0);
    $month  = ($_POST['pay_month'] ?? '') . '-01';
    $method = $_POST['method'] ?? 'UPI';
    $txn    = trim($_POST['txn_ref'] ?? '');

    if (!in_array($propId, array_map('intval',
            array_column($rentals, 'property_id')), true)) {
        $errors[] = 'Select a property you are approved for.';
    }
    if (!preg_match('/^\d{4}-\d{2}$/', $_POST['pay_month'] ?? '')) {
        $errors[] = 'Choose the month you are paying for.';
    }
    if (strlen($txn) < 6) {
        $errors[] = 'Enter a valid transaction reference.';
    }

    if (empty($errors)) {
        $amount = 0.0;
        foreach ($rentals as $r) {
            if ((int) $r['property_id'] === $propId) {
                $amount = (float) $r['rent_amount'];   // authoritative amount
            }
        }

        try {
            $ins = db()->prepare(
                'INSERT INTO payments
                   (property_id, tenant_id, amount, pay_month, method, txn_ref)
                 VALUES (?, ?, ?, ?, ?, ?)'
            );
            $ins->execute([$propId, $uid, $amount, $month, $method, $txn]);

            flash('Rent of ' . money($amount) .
                  ' recorded for ' . month_label($month) . '.');
            redirect('tenant_dashboard.php');

        } catch (PDOException $ex) {
            $errors[] = 'Rent for that month has already been recorded.';
        }
    }
}
?>
```

COMPLAINTS.PHP - MAINTENANCE TRACKING:

```
<?php
require_once __DIR__ . '/includes/functions.php';
require_role('tenant');

$uid = (int) current_user()['user_id'];

$rentals = db()->prepare(
    'SELECT p.property_id, p.title
       FROM applications a
       JOIN properties p ON p.property_id = a.property_id
      WHERE a.tenant_id = ? AND a.status = "approved"'
);
$rentals->execute([$uid]);
$rentals = $rentals->fetchAll();

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    verify_csrf();

    $propId  = (int) ($_POST['property_id'] ?? 0);
    $cat     = $_POST['category'] ?? 'Other';
    $subject = trim($_POST['subject'] ?? '');
    $desc    = trim($_POST['description'] ?? '');
    $prio    = $_POST['priority'] ?? 'Medium';
```

```
    $allowed = array_map(fn($r) => (int) $r['property_id'], $rentals);

    if (!in_array($propId, $allowed, true)) {
        $errors[] = 'Choose one of your approved properties.';
    }
    if (mb_strlen($subject) < 6) {
        $errors[] = 'Subject must be at least 6 characters.';
    }
    if (mb_strlen($desc) < 15) {
        $errors[] = 'Describe the problem in at least 15 characters.';
    }

    if (empty($errors)) {
        $ins = db()->prepare(
            'INSERT INTO complaints
               (property_id, tenant_id, category, subject, description, priority)
             VALUES (?, ?, ?, ?, ?, ?)'
        );
        $ins->execute([$propId, $uid, $cat, $subject, $desc, $prio]);

        flash('Complaint raised. The owner has been notified.');
        redirect('complaints.php');
    }
}
?>
```

APPLICATION_ACTION.PHP - OWNER APPROVAL:

Approving an application is transactional: the application becomes approved, the property becomes occupied and every other pending application for that property is rejected, so a property can never end up with two approved tenants.

```
<?php
require_once __DIR__ . '/includes/functions.php';
require_role('owner');
verify_csrf();

$appId    = (int) ($_POST['application_id'] ?? 0);
$decision = $_POST['decision'] ?? '';

if (!in_array($decision, ['approved', 'rejected'], true)) {
    flash('Invalid decision.', 'error');
    redirect('owner_dashboard.php');
}

/* the application must belong to one of THIS owner's properties */
$check = db()->prepare(
    'SELECT a.application_id, a.property_id, a.status, p.title
       FROM applications a
       JOIN properties p ON p.property_id = a.property_id
      WHERE a.application_id = ? AND p.owner_id = ?'
);
$check->execute([$appId, current_user()['user_id']]);
$app = $check->fetch();

if (!$app || $app['status'] !== 'pending') {
    flash('Application not found or already decided.', 'error');
    redirect('owner_dashboard.php');
}

$pdo = db();
$pdo->beginTransaction();

try {
    $pdo->prepare('UPDATE applications SET status = ? WHERE application_id = ?')
        ->execute([$decision, $appId]);
```

```
    if ($decision === 'approved') {
        $pdo->prepare('UPDATE properties SET status = "occupied" WHERE property_id = ?')
            ->execute([$app['property_id']]);

        $pdo->prepare(
            'UPDATE applications SET status = "rejected"
              WHERE property_id = ? AND status = "pending"
                AND application_id <> ?'
        )->execute([$app['property_id'], $appId]);
    }

    $pdo->commit();
} catch (PDOException $ex) {
    $pdo->rollBack();
    flash('Could not update the application.', 'error');
    redirect('owner_dashboard.php');
}

flash('Application ' . $decision . '.');
redirect('owner_dashboard.php');
```

ASSETS/CSS/STYLE.CSS - KEY RULES:

```
:root{
  --ink:#12 202e;
  --ink-soft:#4b6076;
  --line:#dfe6ee;
  --bg:#f5f7fb;
  --brand:#0f766e;
  --brand-dark:#0b5c56;
  --brand-tint:#e6f4f2;
  --shadow:0 1px 2px rgba(18,32,46,.06), 0 8px 24px rgba(18,32,46,.07);
  --radius:14px;
}

body{
  font-family:"Segoe UI", Roboto, Arial, sans-serif;
  font-size:15px;
  line-height:1.55;
  color:var(--ink);
  background:var(--bg);
}

/* property cards reflow from 3 columns down to 1 */
.cards{
  display:grid;
  grid-template-columns:repeat(auto-fill, minmax(280px,1fr));
  gap:1.2rem;
}

.prop-card{
  background:#fff;
  border:1px solid var(--line);
  border-radius:var(--radius);
  overflow:hidden;
  box-shadow:var(--shadow);
  transition:transform .16s ease, box-shadow .16s ease;
}

.prop-card:hover{
  transform:translateY(−3px);
  box-shadow:0 12px 28px rgba(18,32,46,.13);
}

.prop-thumb{position:relative;aspect-ratio:16/10;overflow:hidden}
.prop-thumb img{width:100%;height:100%;object-fit:cover}

.price-tag{
  position:absolute; left:.7rem; bottom:.7rem;
  background:rgba(18,32,46,.88); color:#fff;
  padding:.3rem .6rem; border-radius:8px;
  font-size:.86rem; font-weight:700;
}

/* status chips */
.chip-green{background:#dcfce7;color:#15 803d}
.chip-blue {background:#e0f2fe;color:#0369a1}
.chip-amber{background:#fef3c7;color:#b45 309}
.chip-red  {background:#fee2e2;color:#b91c1c}

/* sticky header with a CSS-only mobile menu */
.site-header{position:sticky;top:0;z-index:30;background:#fff;
             border-bottom:1px solid var(--line)}
.header-inner{display:flex;align-items:center;
              justify-content:space-between;min-height:66px}

@media (max-width:700px){
  .nav-burger{display:grid;gap:4px;cursor:pointer}
  .site-nav{display:none;flex-direction:column;
            position:absolute;top:66px;left:0;right:0;
```

```
            background:#fff;padding:1rem 4vw}
    .nav-toggle:checked ~ .site-nav{display:flex}
  }
```

ASSETS/JS/MAIN.JS - CLIENT-SIDE VALIDATION:

```
const rules = {
  full_name: v => v.trim().length >= 3 || 'Enter your full name',
  email:     v => /^[^\s@]+@[^\s@]+\.[a-z]{2,}$/i.test(v.trim())
             || 'Enter a valid email address',
  phone:     v => /^[6-9]\d{9}$/.test(v.trim()) || 'Enter a 10-digit mobile',
  password:  v => v.length >= 8 || 'Minimum 8 characters',
  confirm:   v => v === document.getElementById('password').value
             || 'Passwords do not match',
  rent_amount: v => Number(v) > 0 || 'Rent must be greater than zero',
  area_sqft:   v => Number(v) > 0 || 'Area must be greater than zero',
  subject:     v => v.trim().length >= 6 || 'Subject too short',
  description: v => v.trim().length >= 15 || 'Describe the problem'
};

document.querySelectorAll('form[data-validate]').forEach(function (form) {
  form.addEventListener('submit', function (event) {
    let firstBad = null;

    form.querySelectorAll('input, select, textarea').forEach(function (f) {
      const rule = rules[f.name];
      if (!rule) return;

      const result = rule(f.value);
      const ok     = result === true;
      const box    = form.querySelector('[data-error-for="' + f.name + '"]');

      f.classList.toggle('invalid', !ok);
      if (box) {
        box.textContent   = ok ? '' : result;
        box.style.display = ok ? 'none' : 'block';
      }
      if (!ok && !firstBad) firstBad = f;
    });

    if (firstBad) {
      event.preventDefault();
      firstBad.focus();
    }
  });
});

/* live filter over the rendered property cards */
const live = document.getElementById('liveFilter');
if (live) {
  live.addEventListener('input', function () {
    const q = this.value.toLowerCase();
    let shown = 0;

    document.querySelectorAll('.prop-card').forEach(function (card) {
      const hit = (card.dataset.search || '').toLowerCase().includes(q);
      card.style.display = hit ? '' : 'none';
      if (hit) shown++;
    });

    document.getElementById('filterCount').textContent = shown;
  });
}
```

SAMPLE QUERIES USED BY THE APPLICATION:

Purpose	SQL
Login	SELECT * FROM users WHERE email = ? LIMIT 1
Filtered listing	SELECT p.*, u.full_name FROM properties p JOIN users u ON u.user_id = p.owner_id WHERE

	p.status <> 'inactive' AND p.city = ? AND p.bhk = ? ORDER BY p.created_at DESC LIMIT 6 OFFSET 0
Tenant applications	SELECT a.*, p.title FROM applications a JOIN properties p ON p.property_id = a.property_id WHERE a.tenant_id = ? ORDER BY a.applied_on DESC
Owner rent chart	SELECT DATE_FORMAT(pay_month,'%b %Y') AS label, SUM(amount) AS total FROM payments pay JOIN properties p ON p.property_id = pay.property_id WHERE p.owner_id = ? AND pay_month >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH) GROUP BY label ORDER BY pay_month
Complaint queue	SELECT c.*, p.title, u.full_name FROM complaints c JOIN properties p ON p.property_id = c.property_id JOIN users u ON u.user_id = c.tenant_id WHERE p.owner_id = ? ORDER BY FIELD(c.status,'open','in-progress','resolved')

OUTPUT:

The application was started with php -S 127.0.0.1:8088 against a live MySQL 9 instance holding the seed data from database/rentease.sql. The screens below were captured from the running application.

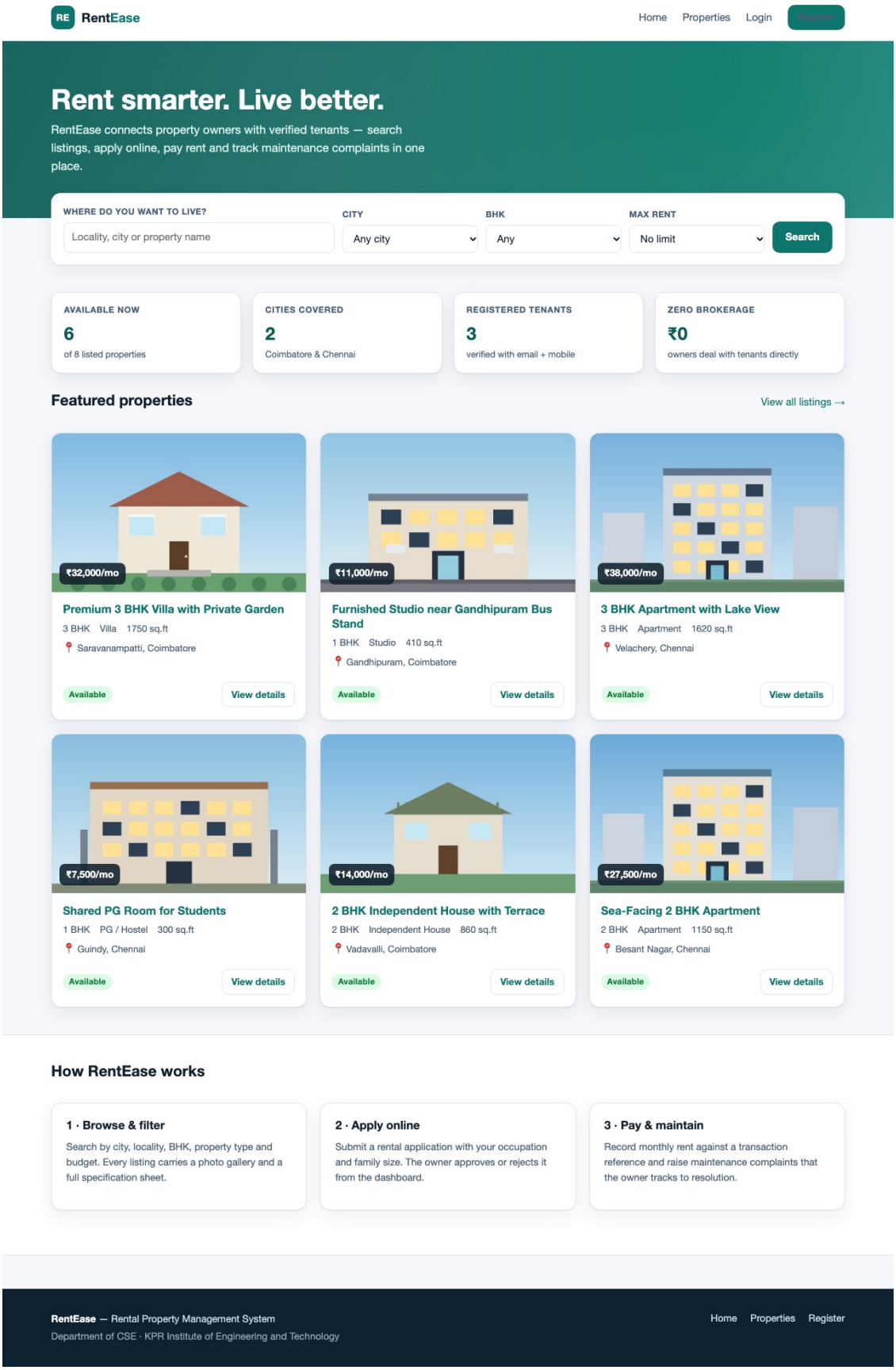


Fig. 9.3 Home page - hero search, portfolio statistics and featured listings

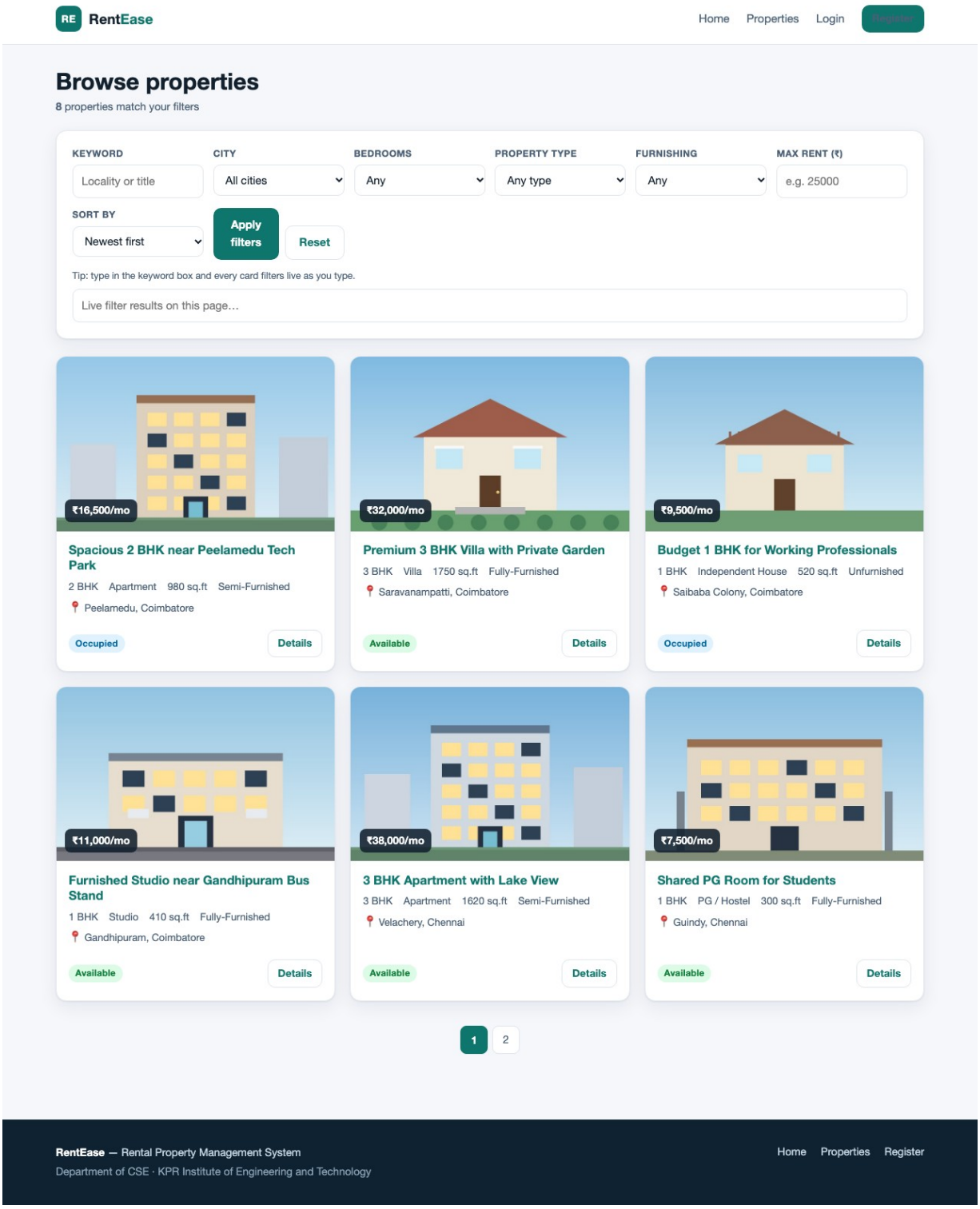


Fig. 9.4 Property listing page with the full filter panel

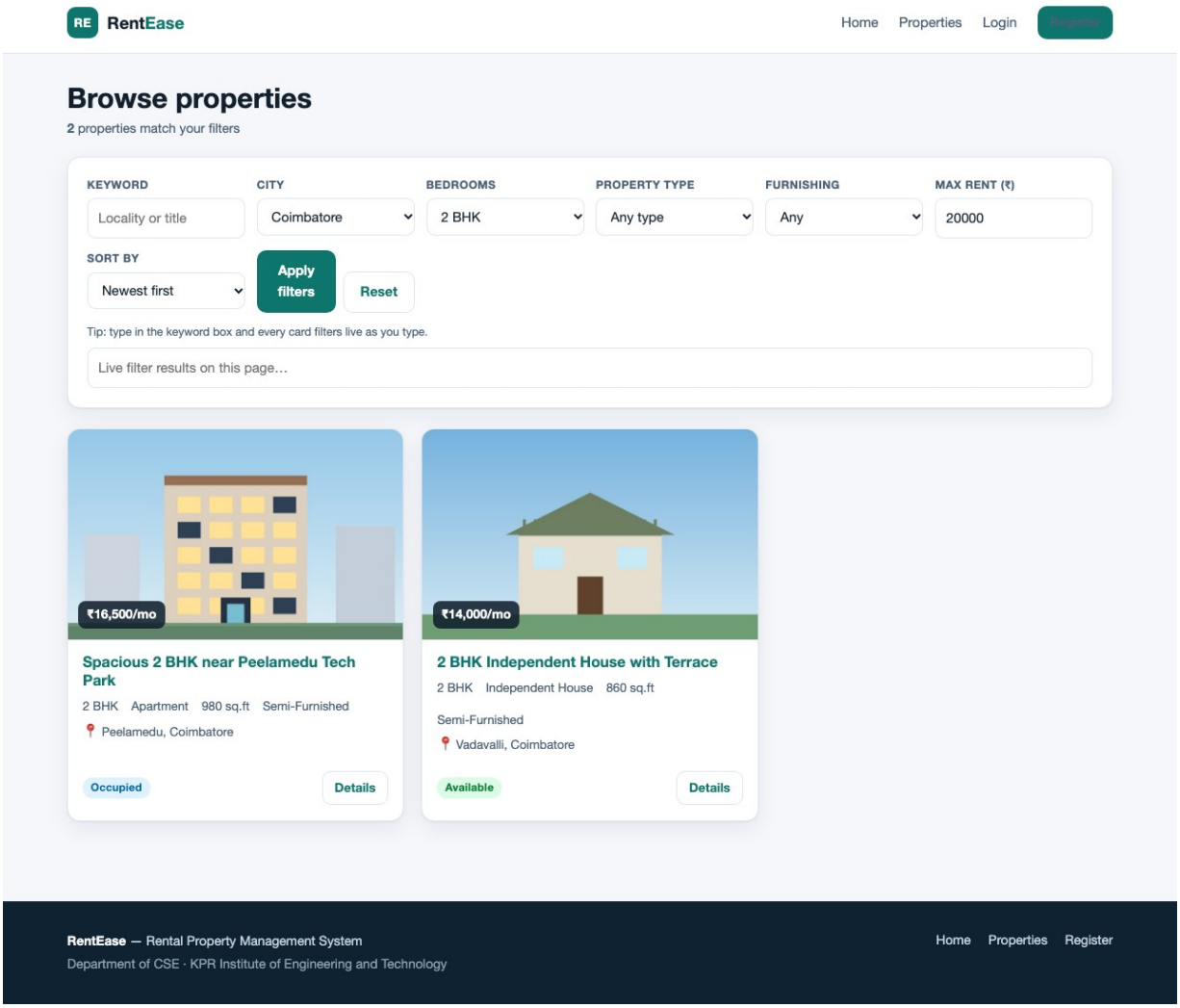


Fig. 9.5 Filter applied (Coimbatore, 2 BHK, under Rs.20,000) - 2 of 8 match

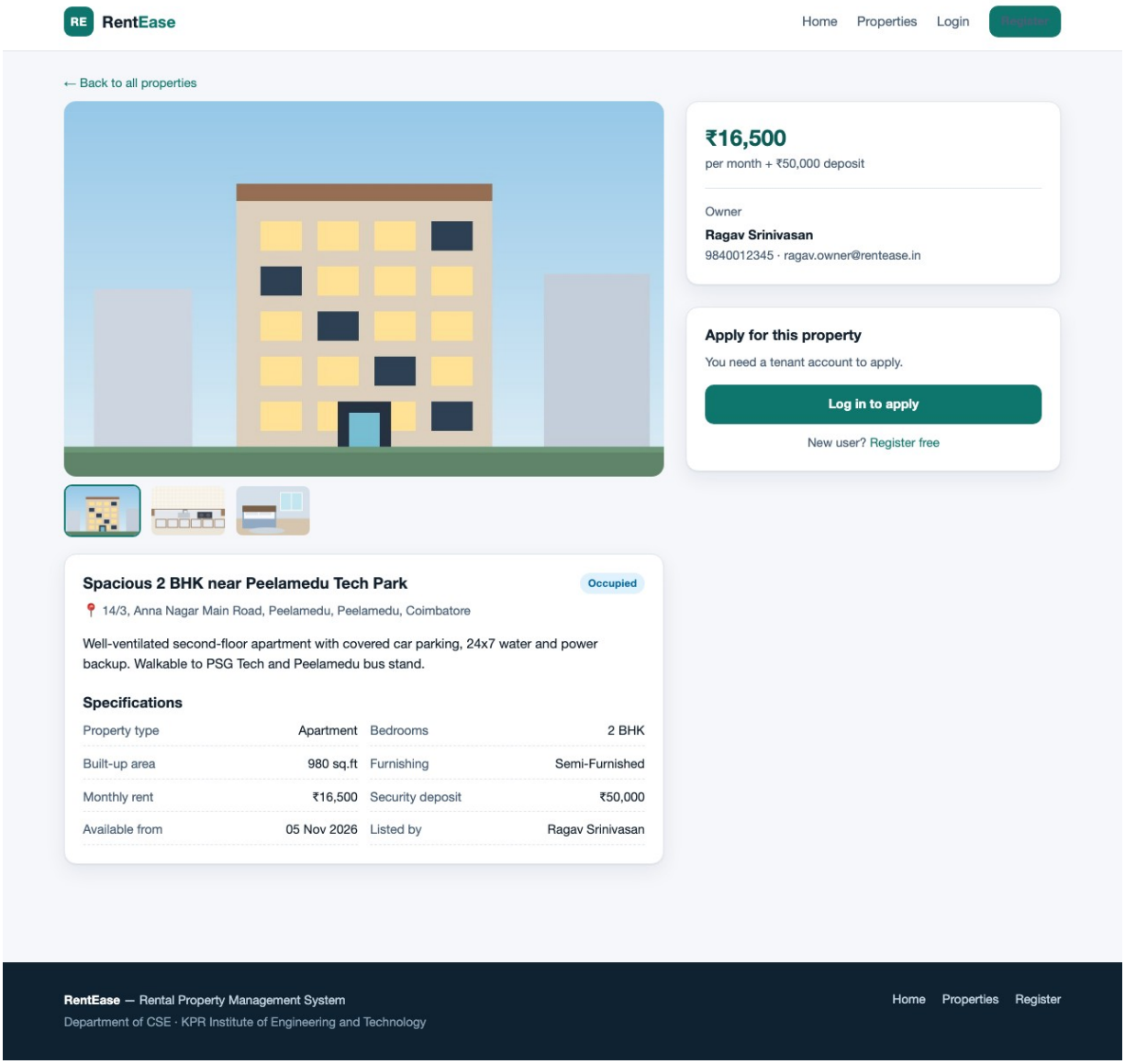


Fig. 9.6 Property detail - gallery, specification sheet and application panel

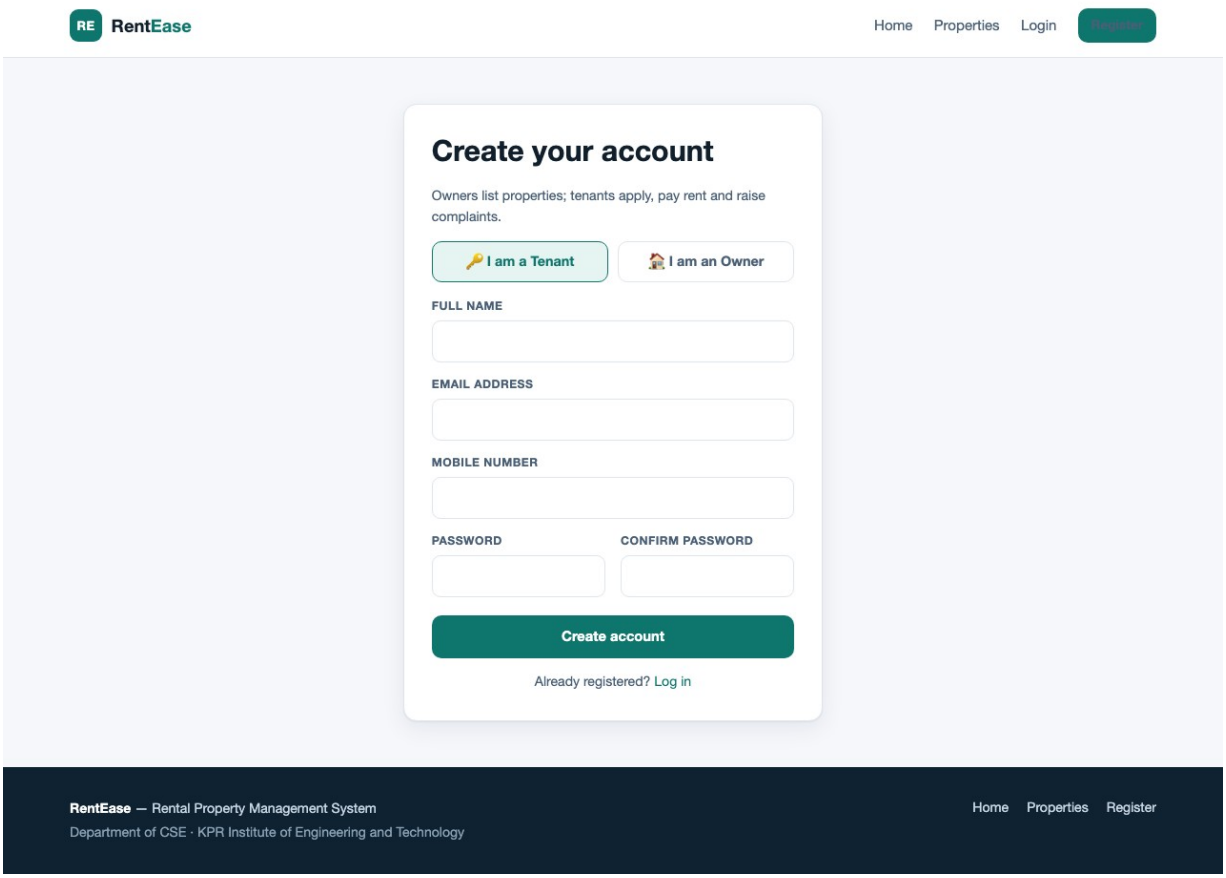


Fig. 9.7 Registration page with the owner / tenant role selector

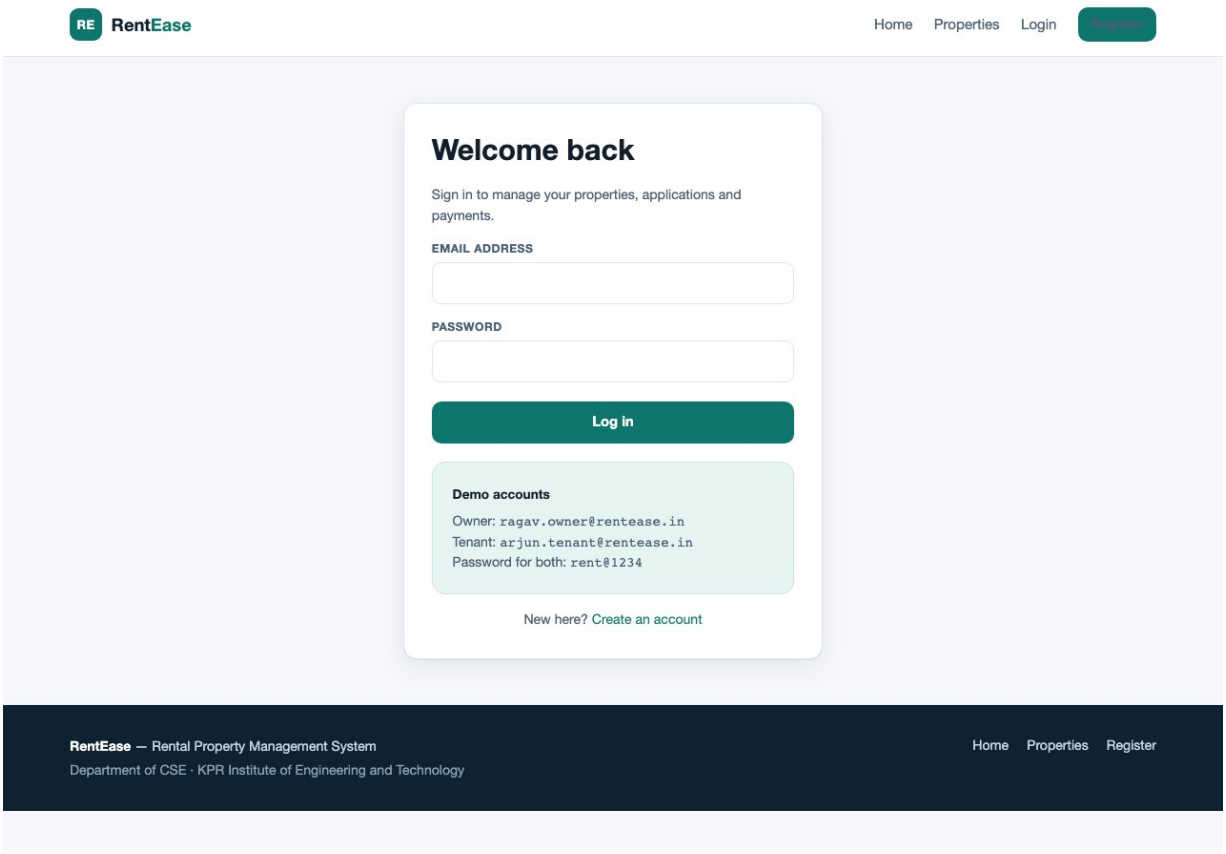


Fig. 9.8 Login page with the demo credentials panel

TENANT WORKFLOWS:

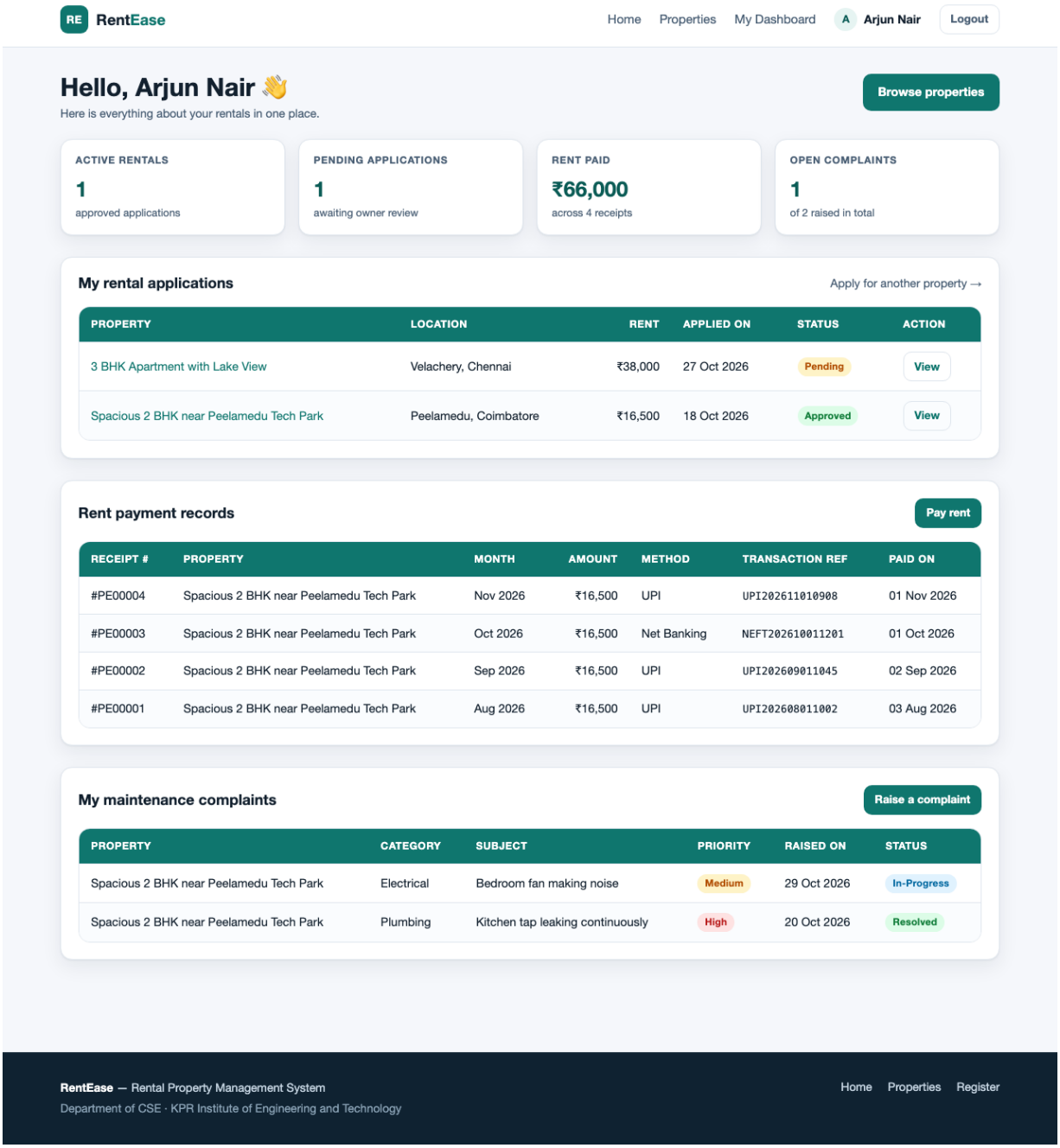


Fig. 9.9 Tenant dashboard - applications, rent receipts and complaint status

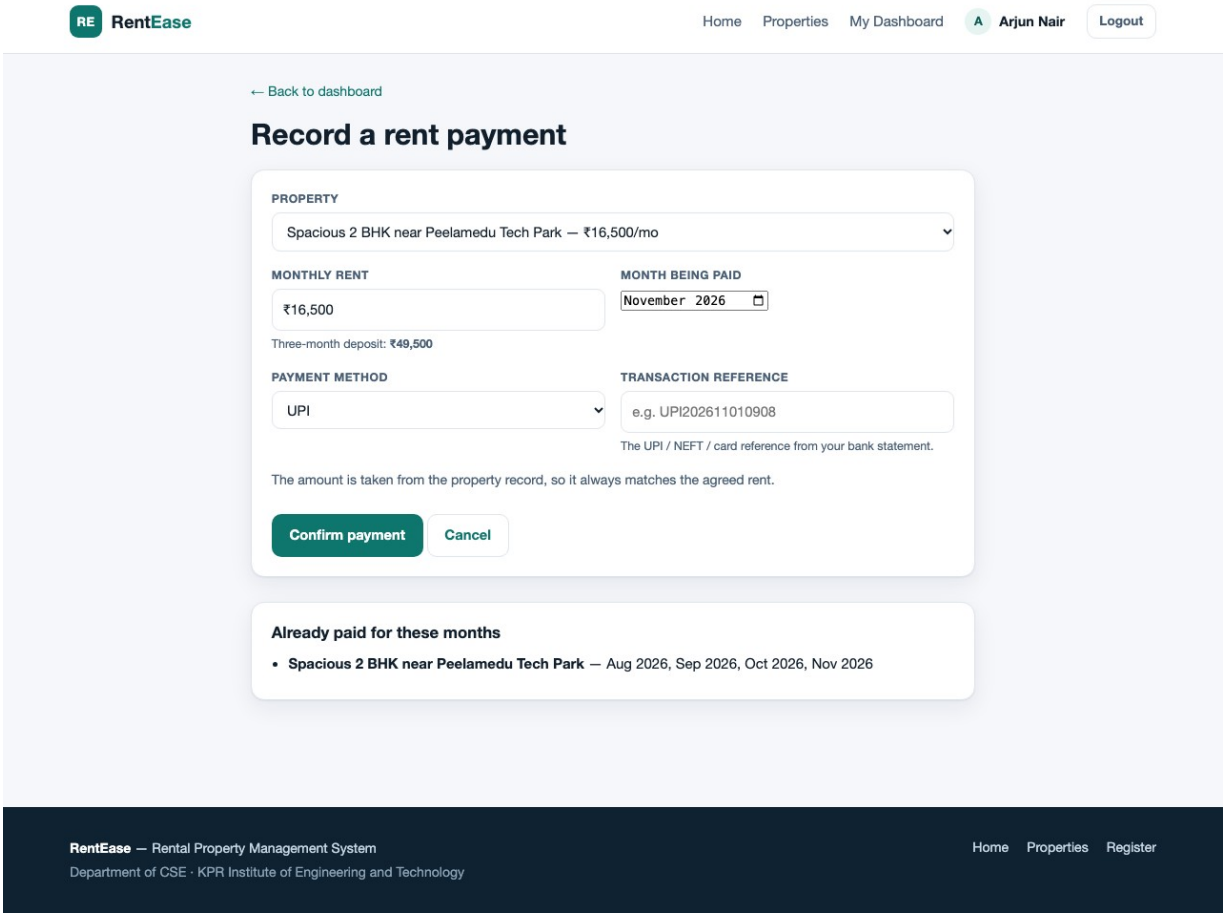


Fig. 9.10 Rent payment form; the amount comes from the property record

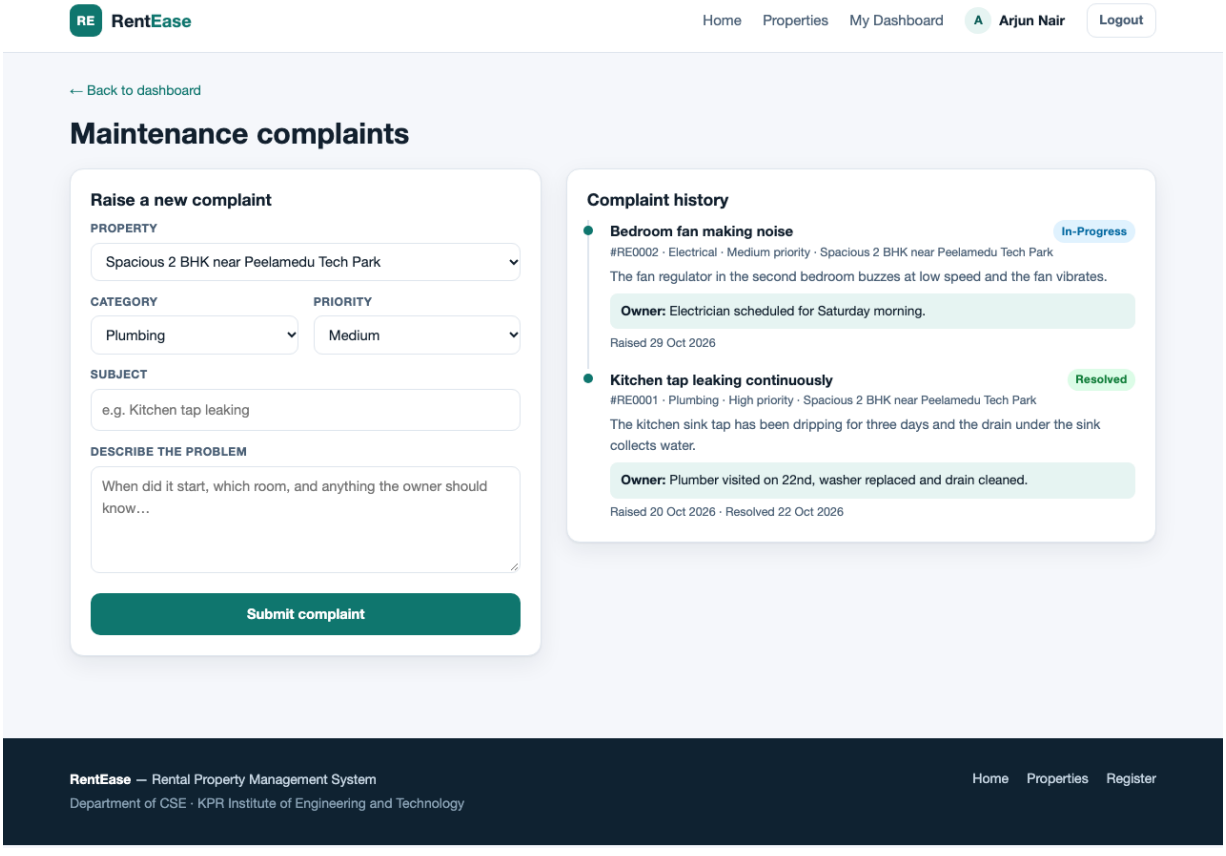


Fig. 9.11 Complaint form with the status timeline and the owner's note

OWNER WORKFLOWS:

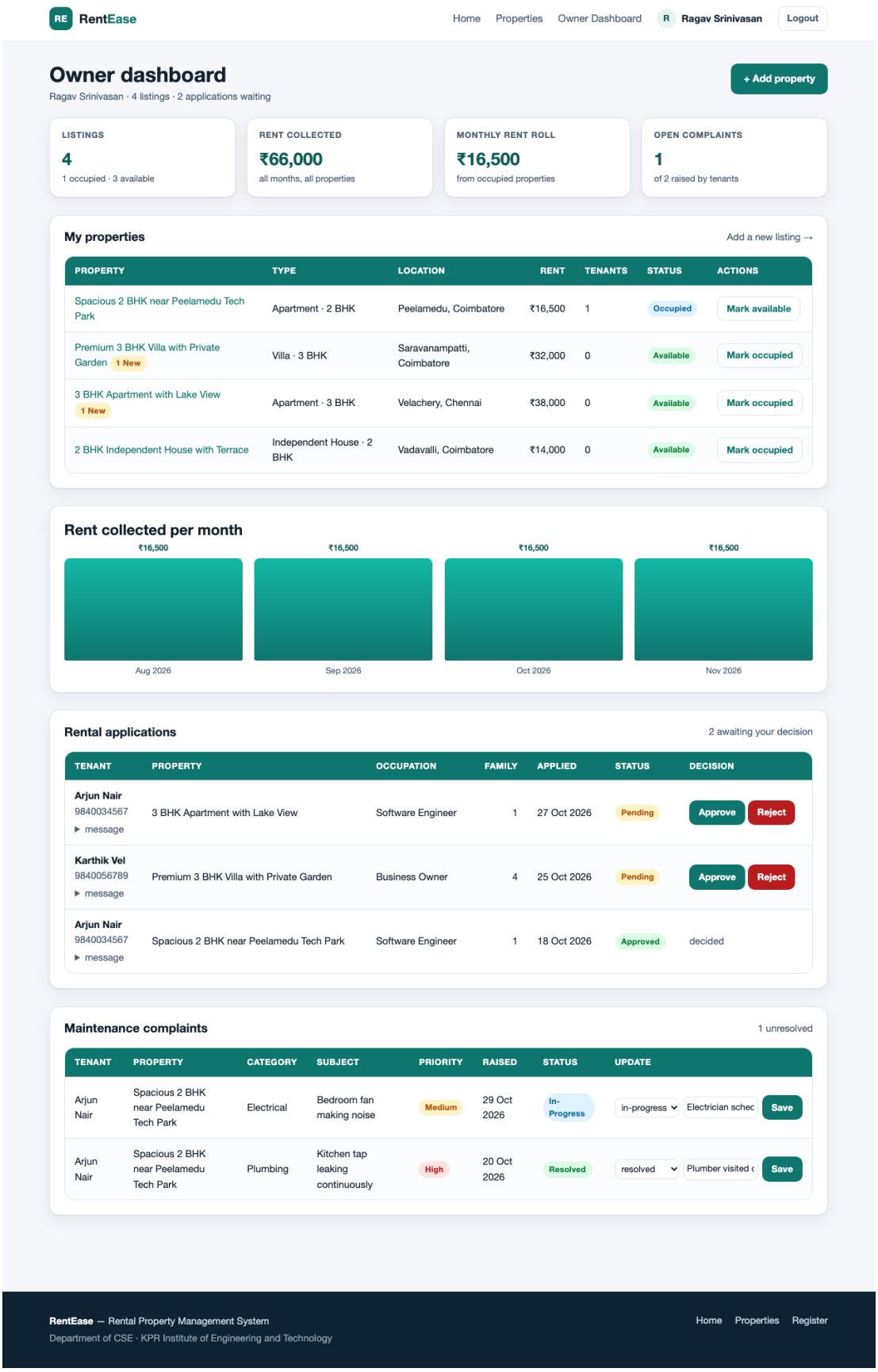


Fig. 9.12 Owner dashboard - portfolio statistics, listings, rent chart, application and complaint queues

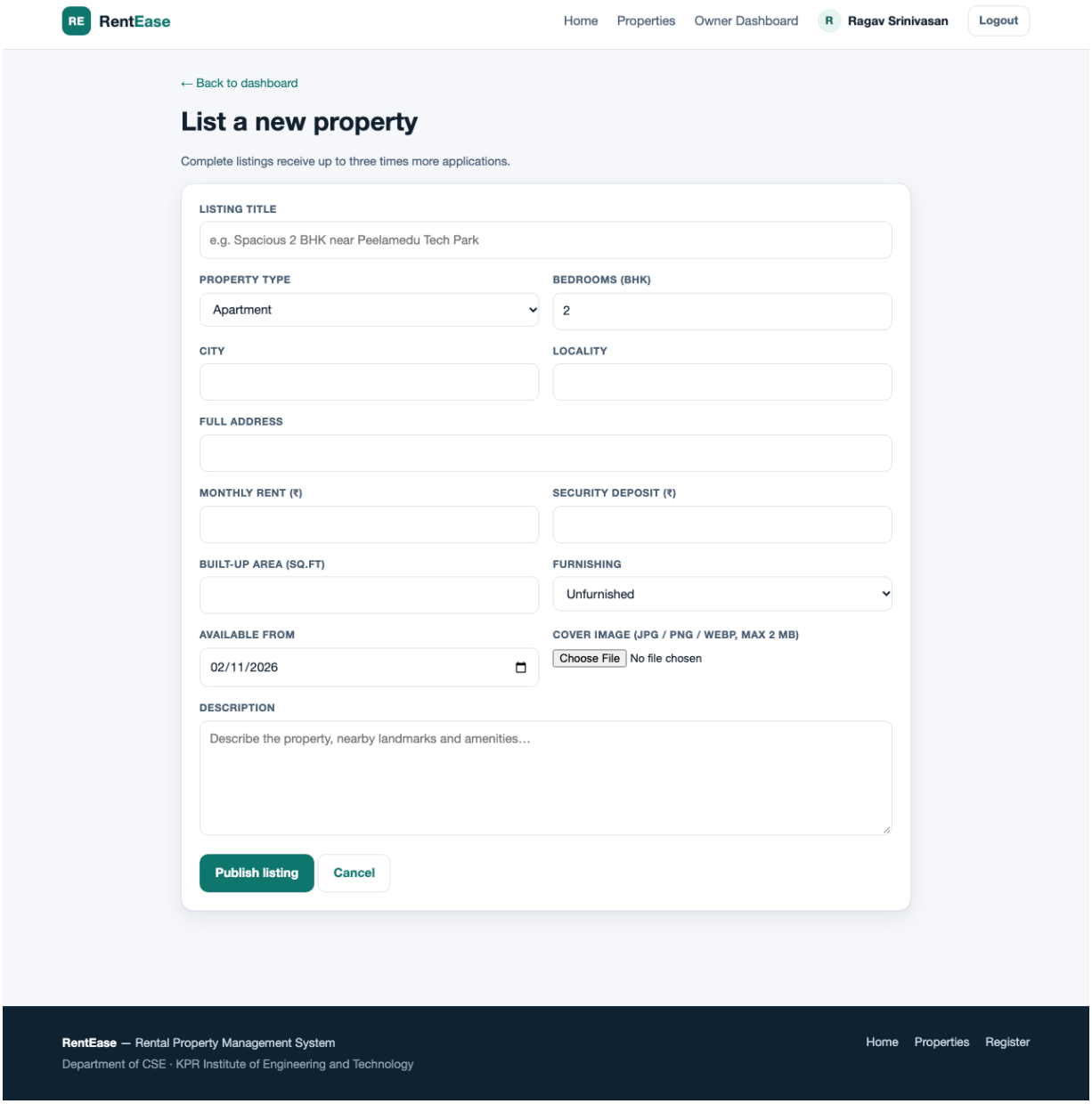


Fig. 9.13 Add property form with cover-image upload

TESTING:

Every workflow was exercised against the live database. The table records the test cases, the expected result and the observed result.

#	Test case	Expected result	Status
1	Register a new tenant	Row inserted, redirected to tenant dashboard	Pass
2	Register with an existing email	"This email is already registered"	Pass
3	Log in with a wrong password	Invalid email or password message	Pass
4	Open a POST form with a bad CSRF token	HTTP 419, request refused	Pass
5	Filter Coimbatore + 2 BHK + max 20,000	2 of 8 listings returned	Pass
6	Tenant applies for a property	Application stored as pending	Pass
7	Tenant applies twice for the same property	Refused, unique key respected	Pass
8	Owner approves an application	Application approved, property occupied	Pass
9	Owner B acts on owner A's application	Refused by the ownership check	Pass
10	Record rent for a new month	Payment row with the property's rent	Pass
11	Record rent twice for one month	Refused, unique key respected	Pass
12	Tenant raises a complaint	Complaint stored as open	Pass
13	Owner marks a complaint resolved	Status resolved, resolved_on set	Pass
14	Owner adds a property with an image	Row plus one gallery image created	Pass
15	Upload a non-image file as cover	Rejected by the MIME check	Pass

RESULT:

Thus the RentEase rental property management system was designed and developed using HTML, CSS, JavaScript, PHP and MySQL. The application supports property listings with image galleries, multi-facet search and filtering, online rental applications with owner approval, month-by-month rent payment records, maintenance complaint tracking and a consolidated owner dashboard. All six features were verified against a live MySQL database and the fifteen test cases above passed.

RUBRICS :

Rubrics	Actual Marks	Awarded Marks
Understanding (R1)	4	
Timeline (R2)	4	
Design (R3)	4	
Development (R4)	4	
Innovation (R5)	4	
Total Marks	**20**	